

**CORREÇÃO ESPECTRAL DO VIÉS
NUMÉRICO NO POTENCIAL
GRAVITACIONAL DE MINI-HALOS COM
PERFIL NFW: DESENVOLVIMENTO E
VALIDAÇÃO DE UM OPERADOR PCHIP PARA
GRADES CARTESIANAS COM DEPOSIÇÃO
CLOUD-IN-CELL V.3**

Jonatas Vitório Silva Ferreira

Critério de classificação estelar/escuro de Nadler et al. (2025)

Área: Astrofísica Computacional / Cosmologia Numérica

Julho de 2026

RESUMO

No modelo cosmológico padrão (Λ CDM), mini-halos de matéria escura ($M \leq 10^8 M_\odot$) constituem os poços de potencial primários para a formação das primeiras galáxias. A profundidade desse poço de potencial, segundo o critério de Nadler et al. (2025), determina se o halo é capaz de reter e resfriar gás bariônico o suficiente para formar estrelas (halo “estelar”) ou permanece escuro. Solvers de Poisson baseados em Transformada Rápida de Fourier (FFT) acoplados a grades cartesianas com deposição *Cloud-in-Cell* (CIC) introduzem, contudo, um viés sistemático de suavização do potencial na região central, com magnitude tipicamente superior a 30% para grades de resolução moderada. Este trabalho quantifica esse viés para halos NFW isolados, comparando a solução numérica (FFT+CIC) com a solução analítica fechada de Łokas e Mamon (2001), e desenvolve um operador de correção não paramétrico bidimensional (concentração $c \times$ raio r), interpolado por *Monotonic Piecewise Cubic Hermite Interpolating Polynomial* (PCHIP), calibrado em uma grade de referência $N = 128$ com 100 translações sub-célula. A correção reduziu o erro médio do potencial de 48,0% para 0,51% (redução de 98,9%) em validação cruzada com grade independente ($N = 64$), e permitiu recuperar corretamente a classificação de 27 dos 27 halos originalmente mal-classificados como “escuros” por viés numérico em uma amostra Monte Carlo de 169 halos (acurácia de 100,0% contra 84,0% do método não corrigido). Testes de robustez revelaram, entretanto, duas limitações importantes: (i) o operador não generaliza para o perfil de Hernquist, produzindo piora de 38% no erro (efeito de sobreajuste ao perfil NFW); e (ii) a classificação binária estelar/escuro mostrou-se sensível à escolha arbitrária do potencial de referência Φ_{ref} , com um mesmo halo trocando de categoria conforme a concentração de referência adotada. Discute-se a escalabilidade computacional do método (expoente empírico $p = 3,76$ versus $p = 3$ teórico) e suas implicações para aplicação em catálogos maiores de halos.

Palavras-chave: Matéria escura. Perfil NFW. Viés numérico. Transformada de Fourier. Cloud-in-Cell. Formação estelar em mini-halos.

Contents

1	Introdução e justificativa	5
1.1	Introdução	5
1.2	Resolvedores de Poisson	7
1.3	Justificativa	8
1.4	Ressalvas e escopo	9
2	Objetivos	9
2.1	Objetivo geral	9
2.2	Objetivos específicos	9
3	Metodologia	10
3.1	Etapa 1 – Solução analítica de referência	10
3.1.1	Perfil de Navarro–Frenk–White (NFW)	10
3.1.2	Perfil de Hernquist	12
3.1.3	Transformação para unidades físicas	14
3.1.4	Potencial de referência e critério de Nadler	14
3.2	Etapa 2 – Solver numérico FFT com deposição	15
3.2.1	Deposição Cloud-in-Cell com subvoxels	16
3.2.2	Solver de Poisson via FFT	17
3.2.3	Orquestração: função <code>phi_num</code>	19
3.3	Etapa 3 – Mapeamento do erro espectral e anisotropia posicional	20
3.3.1	Definição do erro relativo	20
3.3.2	Metodologia de translações aleatórias	20
3.3.3	Estatísticas agregadas do erro	21
3.3.4	Implementação computacional	22
3.4	Etapa 4 – Função de transferência e correção PCHIP	24
3.4.1	Formulação algébrica da correção	24
3.4.2	Interpolação PCHIP no raio	25
3.4.3	Interpolação na concentração	25
3.4.4	Aplicação da correção	26
3.5	Etapa 5 – Validação cruzada	27
3.5.1	Metodologia de comparação	27
3.5.2	Binning radial	28
3.5.3	Métricas de erro	29
3.5.4	Implementação computacional	29
3.6	Etapa 6 – Teste de sensibilidade ao perfil (Hernquist)	31
3.6.1	Metodologia	31

3.6.2	Métricas de eficácia	32
3.6.3	Implementação computacional	32
3.6.4	Resultados esperados e interpretação	33
3.7	Etapa 6.5 – Estudos adicionais de robustez	33
3.7.1	Convergência do número de subvoxels CIC (M_{sub})	34
3.7.2	Sensibilidade ao tamanho da caixa periódica (L)	35
3.7.3	Sensibilidade à escolha da concentração de referência (Φ_{ref})	36
3.7.4	Projeção de escalabilidade computacional	37
3.8	Etapa 7 – Simulação de Monte Carlo	38
3.8.1	Amostragem de massa e concentração	39
3.8.2	Classificação de cada halo	40
3.8.3	Métricas de desempenho	41
3.8.4	Intervalo de confiança de Wilson para a taxa de recuperação	42
3.8.5	Erros do potencial	42
3.9	Etapa 8 – Fração de ocupação de galáxias	44
3.9.1	Definição dos bins de V_{max}	44
3.9.2	Cálculo da fração de ocupação	45
3.9.3	Modelo logístico para a curva de ocupação	45
3.9.4	Interpretação física e validação	46
3.10	Etapa 9 – Geração de figuras	47
3.10.1	Painel [0,0]: Perfis de potencial NFW analíticos	47
3.10.2	Painel [0,1]: Viés $\varepsilon(r)$ antes da correção	47
3.10.3	Painel [1,0]: Validação cruzada	47
3.10.4	Painel [1,1]: Erro antes e depois da correção	47
3.10.5	Painel [2,0]: Scatter Φ_{ana} vs. $\Phi_{\text{num}} / \Phi_{\text{corr}}$	48
3.10.6	Painel [2,1]: Tabela de confusão	48
3.10.7	Estilo visual e salvamento	48
3.11	Etapa 10 – Auditoria e exportação de dados	49
3.11.1	Tabela de auditoria completa	49
3.11.2	Relatório de robustez suplementar	50
3.11.3	Orquestração do pipeline completo	51
4	Resultados	52
4.1	Validação analítica dos perfis NFW e Hernquist	52
4.2	Mapeamento do erro numérico em função da concentração	53
4.3	Validação cruzada do operador de correção	53
4.4	Teste de sensibilidade ao perfil: transferabilidade para Hernquist	53
4.5	Estudos de robustez	54
4.5.1	Convergência em função da subamostragem CIC	54

4.5.2	Sensibilidade ao tamanho da caixa de simulação	54
4.5.3	Fragilidade da classificação estelar/escuro frente ao potencial de referência	55
4.5.4	Escalabilidade computacional	55
4.6	Simulação de Monte Carlo: reclassificação de halos	56
4.7	Impacto na fração de ocupação de galáxias	57
4.8	Figuras e produtos de auditoria gerados	57
5	Discussão	57
6	Limitações	58
7	Conclusão	58
	Referências	60
A	Apêndice A – Ambiente computacional e reprodutibilidade	61

1 Introdução e justificativa

1.1 Introdução

No modelo cosmológico padrão Λ CDM, a formação de estruturas ocorre de maneira hierárquica: mini-halos de matéria escura, com massas iguais ou inferiores a $10^8 M_\odot$, constituem os poços de potencial gravitacional primários necessários ao aprisionamento e resfriamento do gás bariônico primordial. Estudos recentes (NADLER et al., 2025) demonstram que o limiar crítico de massa para a formação estelar nessas estruturas é altamente sensível à profundidade do potencial gravitacional central. Não obstante, as simulações cosmológicas modernas de larga escala recorrem, em sua maioria, a resolvedores numéricos baseados na Transformada Rápida de Fourier (FFT) acoplados a grades cartesianas discretas para a solução da equação de Poisson. Esse processo de discretização espacial, somado ao uso de parâmetros de *softening*¹, introduz inevitavelmente um viés numérico, a suavização artificial da cúspide central do perfil, em escalas próximas ao tamanho da célula da grade.

Antes de iniciarmos as justificativas temos que compreender do que se trata um halo de matéria escura, um halo de matéria escura é definido como uma estrutura de massa estendida e não luminosa que envolve galáxias e aglomerados de galáxias. Diferente das estrelas, que se agrupam em galáxias individuais, a matéria escura distribui-se de forma mais suave, podendo representar dez ou mais vezes a massa das estrelas visíveis em um aglomerado.

Listo aqui algumas definições técnicas:

1. **Perfil de Densidade Universal:** Simulações numéricas (como as de Navarro, Frenk & White - NFW) estabeleceram que os halos possuem um perfil de densidade que varia de r^{-1} próximo ao centro (cúspide) para r^{-3} em grandes distâncias
2. **Composição Não Bariônica:** O halo é composto por matéria que não possui carga elétrica, pois a luz passa por ele sem absorção ou espalhamento, e não emite radiação eletromagnética detectável
3. **Parametrização:** A forma do halo é frequentemente descrita pelo parâmetro de concentração (c), que depende da massa do objeto e do espectro inicial de flutuações de densidade no universo

sendo que a função do halo de matéria escura é atuar como o "andaime gravitacional" do universo, sendo o motor por trás da formação de todas as estruturas visíveis:

1. **Criação de Poços de Potencial:** O halo gera o poço de potencial gravitacional necessário para capturar o gás bariônico (hidrogênio e hélio). Sem a gravidade do halo, as pequenas flutuações de densidade do início do universo não cresceriam rápido o suficiente para formar galáxias.

1

2. **Gatilho para Formação Estelar (Resfriamento):** Uma vez que o gás é aprisionado no poço de potencial do halo, ele colide e irradia energia (processo de resfriamento), permitindo que se concentre no centro e colapse para formar as primeiras estrelas e galáxias.
3. **Berçário das Estrelas de População III:** No início do universo, mini-halos de massas específicas permitiram que o gás primordial atingisse a densidade e temperatura críticas para a ignição nuclear, dando origem à "primeira luz" do cosmos.
4. **Estabilização de Discos Galácticos:** Os halos fornecem o suporte gravitacional onde discos estelares (como os de Miyamoto-Nagai) estão imersos, influenciando diretamente a dinâmica e a dispersão de velocidades das estrelas.
5. **Semente para Buracos Negros:** A ubiquidade de buracos negros supermassivos no centro das galáxias sugere que sua formação está intrinsecamente ligada ao crescimento dos halos através de fusões hierárquicas.

Em resumo, o halo de matéria escura funciona como o gatilho gravitacional que determina se uma região do espaço permanecerá "escura" ou se tornará um berçário estelar visível, dependendo da profundidade e precisão de seu potencial central.

Além disso temos que definir o *Criterio de Nadler* que é um modelo para prever a fração de ocupação de galáxias (f_{gal}) em mini-halos de matéria escura, definindo quais objetos possuem massa suficiente para formar estrelas e quais permanecerão "escuros". Ele se baseia nos seguintes pilares físicos:

1. **Limiar de Resfriamento (M_{Crit}) :** O critério estabelece uma massa crítica dependente do tempo, $M_{Crit}(z)$, necessária para que o gás bariônico sofra resfriamento via hidrogênio molecular (H_2) ou hidrogênio atômico HI .
2. **Histórico de Acreção de Massa (MAH):** Um halo é classificado como "estelar" se o seu histórico de massa exceder o valor de $M_{Crit}(z)$ em qualquer ponto de sua evolução entre o redshift inicial e o momento da observação.
3. **Impacto do Streaming DM-Bárion:** O critério identifica que a velocidade de streaming² entre matéria escura e bárions, impressa na recombinação, é o principal fator que define o limite inferior de formação galáctica, estabelecendo que halos com massa de pico abaixo de $10^7 \odot$ raramente formam estrelas

Com suas respectivas definições do que são halos de matéria escura e do Criterio de Nadler, podemos chegar no núcleo do projeto; os resolvidores de Poisson aplicados a esses halos de matéria escura Via FFT, para finalizar essa introdução então, necessitamos compreender do que se trata os Resolvedores de Poisson em Cosmologia Numérica, junto de seus fundamentos e Limitações.

1.2 Resolvedores de Poisson

1. **A Equação de Poisson no Contexto Gravitacional:** A dinâmica das estruturas no universo, desde mini-halos de matéria escura até aglomerados de galáxias, é governada fundamentalmente pelo potencial gravitacional (ϕ) gerado pela distribuição de massa (ρ). No regime não relativístico, essa relação é descrita pela **Equação de Poisson**:

$$\nabla^2 \phi = 4\pi G \rho$$

onde ∇^2 é o operador Laplaciano e G é a constante gravitacional universal. Para sistemas com geometrias simples, como os perfis esféricos de **Navarro-Frenk-White (NFW)** ou **Hernquist**, existem soluções analíticas fechadas que permitem descrever o potencial em qualquer raio r . Contudo, em simulações cosmológicas de larga escala ou sistemas multi-resolução, a complexidade da distribuição de massa exige o uso de resolvedores numéricos (solvers).

2. **Discretização e Deposição: O Método Cloud-In-Cell (CIC):** A solução numérica começa com a discretização do espaço em uma grade cartesiana de N^3 células. Como a matéria escura e os bárions são frequentemente representados por partículas, é necessário mapear essa massa discreta para os pontos da grade, um processo conhecido como deposição de densidade. O esquema *Cloud-In-Cell* (CIC) é amplamente adotado por equilibrar precisão e custo computacional. Diferente do método *Nearest Grid Point* (NGP), o CIC utiliza uma interpolação linear onde cada partícula é tratada como uma "nuvem" uniforme de massa com o volume de uma célula da grade. A massa é distribuída entre os oito nós vizinhos proporcionalmente ao volume de sobreposição entre a "nuvem" da partícula e cada célula, o que reduz ruídos de discretização em comparação a métodos de ordem inferior.
3. **Resolvedores Espectrais e a Eficiência da FFT:** Para resolver a equação de Poisson na grade, o método mais eficiente baseia-se na Transformada Rápida de Fourier (FFT). A principal vantagem da abordagem espectral é converter o operador diferencial Laplaciano em uma operação algébrica simples no espaço de frequências (k). No espaço de Fourier, a equação de Poisson assume a forma:

$$-k^2 \hat{\phi}(k) = 4\pi G \hat{\rho}(k)$$

Onde $\hat{\phi}$ e $\hat{\rho}$ são as transformadas de Fourier do potencial e da densidade, respectivamente. O potencial é então obtido através da Transformada Inversa de Fourier (IFFT) após a divisão por $-k^2$. Computacionalmente, o uso da FFT reduz a complexidade do problema de $O(N^6)$ para $O(N^3 \log N)$, permitindo a resolução de grades massivas em tempos viáveis.

4. **O Problema da Suavização e Vieses Numéricos:** Apesar de sua eficiência, o par CIC-

FFT introduz artefatos numéricos inevitáveis. A discretização espacial atua como um filtro passa-baixa, suprimindo modos de alta frequência perto da frequência de Nyquist da grade. Esse efeito manifesta-se como uma suavização artificial (smoothing) do potencial central. Em perfis de densidade cuspidos, onde $\rho \propto r^{-1}$ próximo ao centro (como no perfil NFW), o solver numérico falha em capturar a singularidade, resultando em um poço de potencial central mais raso do que a previsão analítica. Além disso, a deposição CIC introduz uma anisotropia posicional: o erro no potencial central depende ligeiramente de onde o centro do halo está localizado em relação aos nós da malha (erro sub-voxel).

5. **Implicações Físicas e Limiares de Ignição:** A precisão do potencial gravitacional central não é apenas uma questão técnica, mas um parâmetro vital para a física da formação galáctica. Segundo critérios modernos de habitabilidade galáctica, como o de Nadler (2025), a profundidade do poço de potencial determina a capacidade de um halo de capturar gás e permitir o resfriamento via hidrogênio molecular (H_2). Subestimar a profundidade desse poço devido a erros de grade pode levar à conclusão errônea de que mini-halos de pequena escala são incapazes de formar estrelas, afetando as previsões sobre a população de estrelas de População III e a cronologia da reionização do universo. Portanto, a auditoria rigorosa desses solvers é o primeiro passo para garantir que as simulações numéricas reflitam a física do universo primitivo.

1.3 Justificativa

A justificativa central deste projeto consiste na necessidade de auditar e quantificar esse viés, propondo um método de correção que impeça que o erro numérico altere artificialmente a fração de ocupação de galáxias na escala de mini-halos. O erro sistemático na profundidade do potencial de mini-halos pode afetar o número previsto de galáxias satélite da Via Láctea, com impacto direto sobre os testes de modelos concorrentes de matéria escura (WIMPs, áxions, matéria escura morna). Com a chegada iminente de dados do *James Webb Space Telescope* e do *Vera C. Rubin Observatory* para galáxias de baixíssima luminosidade, a quantificação e correção desse viés numérico tornam-se urgentes para as comparações entre teoria e observação.

Embora existam propostas de correção de *softening* no espaço real, como o *softening* adaptativo de Iannuzzi e Dolag (2011), ou de deconvolução do potencial para simulações periódicas (GARRISON et al., 2016), nenhum estudo até o momento havia derivado uma função de transferência inversa no espaço de Fourier especificamente calibrada para halos com perfil Navarro-Frenk-White (NFW) em grades cartesianas com deposição *Cloud-in-Cell* (CIC), com o objetivo explícito de corrigir o viés no limiar de formação estelar de mini-halos. Por tanto o foco desse trabalho reside em:

- (i) Quantificar o erro espectral de forma isolada por componente de Fourier e por casca radial;

- (ii) Propor uma correção pré-FFT que, após média sobre 100 translações sub-célula, atenua a dependência do erro em relação à posição do halo na malha;
- (iii) Derivar e comparar uma função de transferência teórica (a partir do kernel CIC, com regularização para altas frequências) e uma função de transferência empírica (PCHIP, validada por leave-one-out);
- (iv) Avaliar o impacto da correção sobre a fração de ocupação de galáxias, uma ponte pouco explorada entre viés numérico puro e astrofísica observacional.

1.4 Ressalvas e escopo

Este projeto <https://github.com/vetorio/Biasnfw/blob/main/biasnfw.py> adota o perfil NFW esférico e isolado como referência contínua, um modelo idealizado. A correção desenvolvida constitui, portanto, uma prova de conceito para halos com simetria esférica; a extensão para halos triaxiais ou para populações cosmológicas reais, com subestrutura e interações, exigirá generalizações futuras, explicitamente fora do escopo deste relatório.

2 Objetivos

2.1 Objetivo geral

Quantificar o erro relativo médio e máximo do potencial gravitacional numérico (via FFT com deposição *Cloud-in-Cell*) em relação à solução analítica contínua para halos NFW esféricos isolados, em função da resolução da grade (N^3) e da concentração c ; e desenvolver um operador de correção no espaço de Fourier, validado contra sobreajuste, que reduza o erro médio para menos de 5% nas escalas radiais ≥ 2 células de grade.

2.2 Objetivos específicos

- (a) Implementar computacionalmente a solução analítica fechada do potencial gravitacional para um halo NFW esférico isolado (ŁOKAS; MAMON, 2001), tomada como referência de exatidão matemática em precisão dupla.
- (b) Desenvolver um solver numérico em Python que deposite a densidade do perfil NFW em grade cartesiana 3D via esquema CIC analítico e resolva a equação de Poisson por FFT (`numpy.fft.rfftn`), com neutralização da densidade média, documentando o erro relativo antes da correção (esperado $> 30\%$ na célula central).
- (c) Mapear o erro relativo do potencial numérico em função da resolução de grade e da concentração c , produzindo tabela de erros médios e máximos.

- (d) Formular uma função de transferência de erro inversa no espaço de Fourier por duas abordagens complementares; teórica (kernel CIC regularizado) e empírica (PCHIP, com validação cruzada *leave-one-out* sobre 100 translações), com critério de sucesso de erro médio $< 5\%$ para $r \geq 2\Delta x$ e $< 15\%$ na célula central.
- (e) Avaliar a ordem de convergência do esquema CIC (esperada de segunda ordem) e a generalidade do método por meio de teste de sensibilidade com o perfil de Hernquist.
- (f) Avaliar o impacto da correção na classificação de formação estelar (critério de Nadler et al., 2025) para uma amostra controlada de halos NFW, com métrica de sucesso de ao menos 3 halos limítrofes reclassificados.

3 Metodologia

A metodologia consistiu no desenvolvimento de um pipeline numérico em Python 3.13.0 (NumPy 2.2.2), executado em ambiente Linux Versão 22.3 (processador AMD64 Family 25 Model 33, 12 CPUs lógicos), organizado em dez etapas sequenciais com métricas de validação explícitas.

3.1 Etapa 1 – Solução analítica de referência

Para estabelecer a verdade fundamental (*ground truth*) do potencial gravitacional contra a qual os resultados numéricos serão comparados, adotamos dois perfis analíticos clássicos para halos de matéria escura: o perfil de Navarro–Frenk–White (NFW) [?] e o perfil de Hernquist [?]. Ambos são implementados no sistema de unidades adimensionais $G = M_{\text{vir}} = r_{\text{vir}} = 1$, onde M_{vir} e r_{vir} são, respectivamente, a massa e o raio viriais do halo. Esta escolha de normalização torna os resultados completamente escaláveis para qualquer massa física, uma vez que a transformação para unidades físicas é realizada a posteriori por fatores de escala dependentes da cosmologia.

3.1.1 Perfil de Navarro–Frenk–White (NFW)

O perfil NFW é caracterizado pela concentração $c = r_{\text{vir}}/r_s$, onde r_s é o raio de escala. No código, a classe `NFWAnalitico` recebe a concentração `c` no construtor e define o raio de escala adimensional como

$$r_s = \frac{1}{c}.$$

Define-se o fator de forma

$$f(c) = \ln(1 + c) - \frac{c}{1 + c},$$

que é calculado no método `__init__` e armazenado no atributo `self.fc`. A densidade característica ρ_s , normalizada para que a massa total dentro do raio virial seja unitária, é dada por

$$\rho_s = \frac{1}{4\pi r_s^3 f(c)}.$$

A densidade radial é então implementada conforme a expressão analítica canônica:

$$\rho(r) = \frac{\rho_s}{\left(\frac{r}{r_s}\right) \left(1 + \frac{r}{r_s}\right)^2}.$$

A massa acumulada dentro do raio r (adimensional) é obtida pela integração direta do perfil de densidade, resultando na expressão fechada

$$M(< r) = \frac{\ln(1 + cr) - \frac{cr}{1+cr}}{f(c)}.$$

No código, esta expressão é implementada no método `massa_acumulada(self, r)`. Nota-se que, para $r = 1$ (raio virial), a expressão se reduz a $M(< 1) = f(c)/f(c) = 1$, garantindo a normalização pretendida.

O potencial gravitacional $\Phi(r)$ para o perfil NFW, conforme Łokas & Mamon (2001), é dado por

$$\Phi(r) = -\frac{\ln(1 + cr)}{r f(c)}.$$

Entretanto, a avaliação direta desta expressão para $r \rightarrow 0$ leva a uma indeterminação do tipo $0/0$. Para contornar este problema numérico, o código implementa no método `potencial(self, r)` uma expansão em série de Taylor de quinta ordem para valores de r inferiores ao limiar `_TAYLOR = 10-6`. Definindo a variável auxiliar $u = cr$, a expansão utilizada é

$$\Phi(r \rightarrow 0) \approx -\frac{c}{f(c)} \left(1 - \frac{u}{2} + \frac{u^2}{3} - \frac{u^3}{4} + \frac{u^4}{5}\right).$$

Para $r \geq 10^{-6}$, a expressão analítica exata é empregada.

A velocidade circular adimensional, $v_c(r) = \sqrt{M(< r)/r}$, é calculada no método `vmax_adim`. O código amostra a função $v_c(r)$ em uma grade linear de 4000 pontos no intervalo $r \in [0.01 r_s, 3.0]$ e retorna o valor máximo encontrado, que corresponde à velocidade máxima adimensional do halo, $v_{\max}^{\text{adim}}$.

A implementação completa da classe é apresentada a seguir.

```
1 class NFWAnalitico:
2     _TAYLOR = 1e-6
3
4     def __init__(self, c: float):
5         self.c = float(c)
6         self.rs = 1.0 / c
```

```

7      self.fc    = np.log(1.0 + c) - c / (1.0 + c)
8      self.rho_s = 1.0 / (4.0 * np.pi * self.rs**3 * self.fc)
9      self.nome  = "NFW"
10
11     def densidade(self, r):
12         r = np.asarray(r, dtype=float)
13         x = np.where(r < 1e-14, 1e-14, r) / self.rs
14         return self.rho_s / (x * (1.0 + x)**2)
15
16     def potencial(self, r):
17         r = np.asarray(r, dtype=float)
18         phi = np.empty_like(r)
19         t = r < self._TAYLOR
20         re = r[~t]
21         if re.size:
22             phi[~t] = -np.log(1.0 + self.c * re) / (re * self.fc)
23         rt = r[t]
24         if rt.size:
25             u = self.c * rt
26             phi[t] = -(self.c / self.fc) * (
27                 1.0 - u/2 + u**2/3 - u**3/4 + u**4/5)
28         return phi
29
30     def massa_acumulada(self, r):
31         cr = self.c * np.asarray(r, dtype=float)
32         return (np.log(1.0 + cr) - cr / (1.0 + cr)) / self.fc
33
34     def vmax_adim(self):
35         r = np.linspace(0.01 * self.rs, 3.0, 4000)
36         vc = np.sqrt(self.massa_acumulada(r) / r)
37         return float(vc.max())
38
39     def __repr__(self):
40         return f"NFWAnalitico(c={self.c:.1f})"

```

Listing 1: Classe NFWAnalitico

3.1.2 Perfil de Hernquist

O perfil de Hernquist, assim como o NFW, apresenta uma cúspide interna $\rho \propto r^{-1}$, mas possui um potencial gravitacional analiticamente mais simples, sem dependência de funções transcendentais, o que facilita sua utilização em alguns contextos dinâmicos.

A classe `HernquistAnalitico` é implementada com a mesma normalização $G = M_{\text{vir}} = r_{\text{vir}} = 1$. O parâmetro de escala a é definido no construtor como

$$a = \frac{1}{c \cdot 2.16},$$

onde c é a concentração equivalente ao NFW. O código utiliza, portanto, a relação aproximada $a \approx r_{\text{vir}}/(c \cdot 2.16)$ para equiparar o raio de escala ao do perfil NFW.

A densidade é expressa por

$$\rho(r) = \frac{a}{2\pi r (r + a)^3},$$

a massa acumulada é

$$M(< r) = \frac{r^2}{(r + a)^2},$$

e o potencial gravitacional, notavelmente simples e sem singularidade numérica no centro, é dado por

$$\Phi(r) = -\frac{1}{r + a}.$$

O método `vmax_adim` realiza o mesmo procedimento numérico descrito para o NFW: calcula $v_c(r) = \sqrt{M(< r)/r}$ em uma grade e retorna o máximo.

```

1 class HernquistAnalitico:
2     def __init__(self, c: float):
3         self.c      = float(c)
4         self.a      = 1.0 / (c * 2.16)
5         self.nome   = "Hernquist"
6
7     def densidade(self, r):
8         r = np.asarray(r, dtype=float)
9         r = np.where(r < 1e-14, 1e-14, r)
10        a = self.a
11        return 1.0 / (2.0 * np.pi) * a / (r * (r + a)**3)
12
13    def potencial(self, r):
14        r = np.asarray(r, dtype=float)
15        return -1.0 / (np.where(r < 1e-14, 1e-14, r) + self.a)
16
17    def massa_acumulada(self, r):
18        r = np.asarray(r, dtype=float)
19        return r**2 / (r + self.a)**2
20
21    def vmax_adim(self):
22        r = np.linspace(0.001 * self.a, 3.0, 4000)
23        vc = np.sqrt(self.massa_acumulada(r) / r)
24        return float(vc.max())
25
26
27    def __repr__(self):
28        return f"HernquistAnalitico(c={self.c:.1f}, a={self.a:.4f})"

```

Listing 2: Classe `HernquistAnalitico`

3.1.3 Transformação para unidades físicas

Para conectar as soluções adimensionais a halos cosmológicos reais, o código implementa funções de escalonamento que dependem dos parâmetros cosmológicos definidos nas constantes globais da Seção A.

A densidade crítica do Universo é definida como

$$\rho_{\text{crit}} = 2.775 \times 10^{11} h^2 M_{\odot} \text{Mpc}^{-3},$$

onde $h = 0.70$. A sobredensidade virial é fixada em $\Delta_c = 200$. Para uma dada massa virial M_{vir} (em massas solares), o raio virial físico é calculado por

$$r_{\text{vir}}(M_{\text{vir}}) = \left(\frac{3M_{\text{vir}}}{4\pi\Delta_c\rho_{\text{crit}}} \right)^{1/3}.$$

A velocidade virial correspondente é

$$V_{\text{vir}} = \sqrt{\frac{GM_{\text{vir}}}{r_{\text{vir}}}},$$

onde a constante gravitacional em unidades compatíveis é $G = 4.302 \times 10^{-3} \times 10^{-6} \text{Mpc } M_{\odot}^{-1} (\text{km/s})^2$.

Estas operações são realizadas pela função `Mvir_para_Vvir`:

```
1 def Mvir_para_Vvir(M_sol):
2     r_vir = (3.0 * M_sol / (4.0 * np.pi * DELTA_C * RHO_CRIT))**(1/3)
3     return float(np.sqrt(G_MPC * M_sol / r_vir))
```

A velocidade máxima física do halo é obtida simplesmente escalonando a velocidade máxima adimensional pela velocidade virial:

$$V_{\text{max}}^{\text{fsico}} = v_{\text{max}}^{\text{adim}} \times V_{\text{vir}}.$$

Esta operação é implementada na função `Vmax_fisico`:

```
1 def Vmax_fisico(perfil, V_vir_kms):
2     return perfil.vmax_adim() * V_vir_kms
```

3.1.4 Potencial de referência e critério de Nadler

O critério de classificação estelar utilizado neste trabalho segue a prescrição de Nadler *et al.* (2025). Para a sua aplicação, é necessário definir um potencial de referência Φ_{ref} contra o qual o potencial do halo em análise será comparado.

A constante global `PHI_REF` é calculada no momento da importação do módulo. O código cria uma instância do perfil NFW com concentração $c_{\text{ref}} = 10$ e avalia o seu potencial analítico

no raio $R_{\text{eval}} = 0.10$ (em unidades do raio virial):

$$\Phi_{\text{ref}} = \Phi_{\text{NFW}}(R_{\text{eval}}; c = 10).$$

```
1 _NFW_REF = NFWAnalitico(10)
2 PHI_REF = float(_NFW_REF.potencial(np.array([R_EVAL]))[0])
```

Define-se então o *score* de profundidade do poço de potencial, S , como a razão entre o módulo do potencial do halo no raio R_{eval} e o módulo do potencial de referência:

$$S = \frac{|\Phi(R_{\text{eval}})|}{|\Phi_{\text{ref}}|}.$$

Esta operação é implementada na função `score_phi`:

```
1 def score_phi(phi_val, phi_ref):
2     return abs(phi_val) / abs(phi_ref)
```

O critério dual de Nadler é então definido pela conjunção de duas condições: (i) $S > \text{SCORE_LIM} = 0.85$ e (ii) $V_{\text{max}} > \text{VMAX_LIM_KMS} = 12.0 \text{ km/s}$. Este critério é encapsulado na função `critério_nadler`:

```
1 def critério_nadler(phi_r_eval, Vmax_kms):
2     sc = score_phi(phi_r_eval, PHI_REF)
3     return (sc > SCORE_LIM) and (Vmax_kms > VMAX_LIM_KMS)
```

A solução analítica descrita nesta etapa serve como referência exata para a validação dos métodos numéricos implementados nas etapas seguintes, permitindo a quantificação do viés introduzido pela discretização espacial e a aferição da eficácia do corretor PCHIP proposto.

3.2 Etapa 2 – Solver numérico FFT com deposição

Uma vez estabelecida a solução analítica de referência, a etapa seguinte consiste em obter uma representação numérica do potencial gravitacional a partir do perfil de densidade contínuo. Para tanto, o código implementa um solver de Poisson baseado em transformada rápida de Fourier (FFT), precedido por um esquema de deposição *Cloud-in-Cell* (CIC) com subdivisão volumétrica. A implementação é inteiramente realizada em unidades adimensionais ($G = M_{\text{vir}} = r_{\text{vir}} = 1$) sobre o domínio cúbico $[-L/2, +L/2]^3$, com $L = \text{CAIXA_L} = 2.0$, ou seja, $r \in [-1, 1]$.

O solver é composto por três componentes principais: (i) a deposição da densidade contínua em uma grade tridimensional, (ii) a resolução da equação de Poisson no espaço de Fourier e (iii) a correção das condições de contorno periódicas inerentes à FFT.

3.2.1 Deposição Cloud-in-Cell com subvoxels

A grade de densidade é representada por um arranjo tridimensional de tamanho $N \times N \times N$, onde cada célula tem lado

$$\Delta x = \frac{L}{N}.$$

As coordenadas dos centros das células são dadas por

$$x_i = -1 + \left(i + \frac{1}{2}\right) \Delta x, \quad i = 0, \dots, N-1,$$

e analogamente para as direções y e z .

Para evitar o *aliasing*³ introduzido pela simples amostragem do perfil no centro de cada célula, especialmente na região da cúspide central onde a densidade varia rapidamente, o código emprega um esquema de subdivisão: cada célula é particionada em M_{sub}^3 subvoxels de lado

$$\Delta x_s = \frac{\Delta x}{M_{\text{sub}}}.$$

As posições relativas dos subvoxels dentro da célula são definidas pelo vetor

$$\delta_\alpha = \left(\alpha + \frac{1}{2}\right) \Delta x_s - \frac{\Delta x}{2}, \quad \alpha = 0, \dots, M_{\text{sub}} - 1.$$

Para um halo deslocado por um vetor $\mathbf{d} = (d_x, d_y, d_z)$, a densidade na célula de índices (i, j, k) é calculada como a média aritmética das densidades avaliadas em todos os subvoxels que a compõem:

$$\rho_{ijk} = \frac{1}{M_{\text{sub}}^3} \sum_{\alpha=0}^{M_{\text{sub}}-1} \sum_{\beta=0}^{M_{\text{sub}}-1} \sum_{\gamma=0}^{M_{\text{sub}}-1} \rho \left(\begin{array}{c} x_i + \delta_\alpha - d_x, \\ y_j + \delta_\beta - d_y, \\ z_k + \delta_\gamma - d_z \end{array} \right),$$

onde $\rho(\mathbf{r})$ é a função densidade analítica do perfil (NFW ou Hernquist). Na implementação, este processo é realizado iterativamente sobre os subvoxels, acumulando as contribuições em uma única grade tridimensional `rho` e evitando a alocação de um tensor 6D completo. Este cuidado é essencial para a viabilidade computacional: para $N = 128$ e $M_{\text{sub}} = 8$, um tensor 6D ingênuo exigiria cerca de 512 GB de memória, enquanto o esquema iterativo mantém o pico de uso em aproximadamente $2 \times N^3 \times 8$ bytes, ou cerca de 34 MB para $N = 128$.

A implementação completa da deposição CIC é apresentada a seguir.

```
1 def cic_loop(perfil, N, deslocamento, M_sub=8):
2     dx      = CAIXA_L / N
3     dx_s    = dx / M_sub
4     coords = -1.0 + (np.arange(N) + 0.5) * dx
```

```

5     off    = (np.arange(M_sub) + 0.5) * dx_s - dx / 2.0
6     peso   = (dx_s / dx)**3
7     cx, cy, cz = deslocamento
8     rho = np.zeros((N, N, N), dtype=np.float64)
9
10    nome = getattr(perfil, 'nome', 'perfil')
11    print(f" [CIC-{nome}] N={N}, M_sub={M_sub} ({M_sub**3}sv/cel) ...",
12          end=" ", flush=True)
13    t0 = time.perf_counter()
14
15    for si in off:
16        xi = coords + si - cx
17        for sj in off:
18            yj = coords + sj - cy
19            for sk in off:
20                zk = coords + sk - cz
21                r3d = np.sqrt(xi[:,None,None]**2
22                              + yj[None,:,None]**2
23                              + zk[None,None,:]**2)
24                rho += perfil.densidade(r3d) * peso
25
26    print(f"OK ({time.perf_counter()-t0:.1f}s)")
27    return rho

```

Listing 3: Função `cic_loop` para deposição CIC com subvoxels

3.2.2 Solver de Poisson via FFT

A equação de Poisson no sistema de unidades adotado é

$$\nabla^2 \Phi(\mathbf{r}) = 4\pi \rho(\mathbf{r}),$$

com $\rho(\mathbf{r})$ representando a densidade numérica depositada na grade. Aplicando a transformada de Fourier contínua (definida como $\hat{f}(\mathbf{k}) = \int f(\mathbf{r})e^{-i\mathbf{k}\cdot\mathbf{r}}d^3r$), o operador Laplaciano se torna uma multiplicação por $-k^2$, de modo que a equação no espaço de Fourier se reduz a

$$\hat{\Phi}(\mathbf{k}) = -\frac{4\pi\hat{\rho}(\mathbf{k})}{k^2},$$

onde $k^2 = k_x^2 + k_y^2 + k_z^2$.

O código utiliza a função `np.fft.rfftn` para realizar a transformada real-complexa da grade de densidade, explorando a simetria de funções reais para reduzir pela metade o número de modos de Fourier na última dimensão. Para garantir a convergência da transformada, a densidade média da grade é subtraída antes da transformação, i.e., calcula-se $\rho'(\mathbf{r}) = \rho(\mathbf{r}) - \langle \rho \rangle$. Esta operação equivale a adicionar um fundo uniforme de densidade negativa e é uma prática

padrão em simulações com condições de contorno periódicas, evitando a divergência do modo $k = 0$.

Os números de onda em cada direção são definidos como

$$k_x = \frac{2\pi}{L} \cdot \text{fftfreq}(N), \quad k_y = \frac{2\pi}{L} \cdot \text{fftfreq}(N), \quad k_z = \frac{2\pi}{L} \cdot \text{rfftfreq}(N),$$

onde as funções `fftfreq` e `rfftfreq` geram as frequências apropriadas para as transformadas FFT e RFFT, respectivamente. A norma ao quadrado do vetor de onda é então calculada como

$$k^2 = k_x[:, \text{None}, \text{None}]^2 + k_y[\text{None}, :, \text{None}]^2 + k_z[\text{None}, \text{None}, :]^2.$$

O potencial no espaço de Fourier é obtido por

$$\hat{\Phi}(k_x, k_y, k_z) = -\frac{4\pi\hat{\rho}'(k_x, k_y, k_z)}{k^2},$$

com o cuidado especial de evitar a divisão por zero no modo $k = 0$: neste modo, o valor de $\hat{\Phi}$ é explicitamente definido como zero, fixando assim a constante aditiva do potencial.

A transformada inversa é realizada pela função `np.fft.irfftn`, com a especificação explícita dos eixos de transformação `axes=(0, 1, 2)` para garantir compatibilidade com versões recentes do NumPy (a partir da versão 2.0), eliminando `DeprecationWarning`.

Um ponto crítico da implementação é a correção das condições de contorno. A FFT impõe naturalmente periodicidade à solução, o que é inadequado para um halo isolado, cujo potencial deveria tender a zero no infinito. Para contornar esta limitação, o código adota uma correção empírica: calcula-se o raio de cada ponto da grade em relação ao centro da caixa:

$$r_{\text{grid}}(i, j, k) = \sqrt{x_i^2 + y_j^2 + z_k^2},$$

e seleciona-se uma casca esférica externa definida por

$$r_{\text{grid}} > 0.85 \cdot \sqrt{3},$$

onde $\sqrt{3}$ é a distância do centro até o vértice da caixa cúbica $[-1, 1]^3$. Calcula-se então a média de Φ nesta casca externa e subtrai-se este valor de toda a grade, forçando o potencial a ter média zero nas bordas e aproximando, assim, a condição de contorno de Dirichlet $\Phi \rightarrow 0$ para $r \rightarrow \infty$.

A função `poisson_fft` implementa todo este procedimento.

```
1 def poisson_fft(rho, L=CAIXA_L):
2     N = rho.shape[0]
3     dx = L / N
4     print(f"      [FFT] Poisson N={N} ...", end=" ", flush=True)
5     t0 = time.perf_counter()
6
```

```

7   rho_k = np.fft.rfftn(rho - rho.mean())
8   kx     = (2*np.pi/L) * np.fft.fftfreq(N, d=1.0/N)
9   kz     = (2*np.pi/L) * np.fft.rfftfreq(N, d=1.0/N)
10  k2      = kx[:,None,None]**2 + kx[None,:,None]**2 + kz[None,None,:]**2
11  k2s     = k2.copy(); k2s[0,0,0] = 1.0
12  phi_k   = -4.0 * np.pi * rho_k / k2s
13  phi_k[0,0,0] = 0.0
14
15  phi = np.fft.irfftn(phi_k, s=(N, N, N), axes=(0, 1, 2))
16
17  coords = -1.0 + (np.arange(N) + 0.5) * dx
18  rg = np.sqrt(coords[:,None,None]**2
19             + coords[None,:,None]**2
20             + coords[None,None,:]**2)
21  borda = rg > 0.85 * np.sqrt(3.0)
22  if borda.any():
23      phi -= phi[borda].mean()
24
25  print(f"OK ({time.perf_counter()-t0:.1f}s)")
26  return phi

```

Listing 4: Função `poisson_fft` para solução da equação de Poisson via FFT

3.2.3 Orquestração: função `phi_num`

A função `phi_num` atua como um orquestrador que combina as duas etapas anteriores (deposição CIC e solver de Poisson) em um único fluxo de trabalho. Para um dado perfil analítico, resolução N , deslocamento e número de subvoxels, a função:

1. Calcula o espaçamento da grade $\Delta x = L/N$;
2. Invoca `cic_loop` para obter a grade de densidade ρ ;
3. Invoca `poisson_fft` para resolver a equação de Poisson, obtendo o potencial Φ na grade;
4. Libera a memória da grade de densidade (`del rho`) e aciona o coletor de lixo (`gc.collect()`) para minimizar o pico de uso de memória;
5. Calcula a grade de distâncias radiais r_{grid} em relação ao centro do sub-halo definido pelo vetor deslocamento, utilizando a mesma fórmula de broadcasting tridimensional.

O resultado é um par (Φ, r_{grid}) , onde ambos são arrays tridimensionais de tamanho N^3 . O potencial retornado está em unidades adimensionais de V_{vir}^2 , compatível com o potencial analítico da Etapa 1.

```

1 def phi_num(perfil, N, deslocamento, M_sub=8):
2     dx = CAIXA_L / N
3     rho = cic_loop(perfil, N, deslocamento, M_sub)
4     phi = poisson_fft(rho)
5     del rho; gc.collect()
6     cx, cy, cz = deslocamento
7     coords = -1.0 + (np.arange(N) + 0.5) * dx
8     rg = np.sqrt((coords-cx)[: ,None,None]**2
9                 + (coords-cy)[None, :,None]**2
10                + (coords-cz)[None,None, :]**2)
11     return phi, rg

```

Listing 5: Função `phi_num` para orquestração do solver numérico

A solução numérica obtida por este pipeline será comparada com a solução analítica de referência nas etapas subsequentes, permitindo a quantificação do viés introduzido pela discretização e a subsequente correção via PCHIP.

3.3 Etapa 3 – Mapeamento do erro espectral e anisotropia posicional

A solução numérica obtida pela deposição CIC seguida do solver de Poisson via FFT introduz dois tipos de erro sistemático em relação à solução analítica exata: (i) um erro dependente do raio, denominado *erro espectral*, que decorre da resolução finita da grade e da discretização da cúspide central; e (ii) um erro dependente da orientação relativa entre o centro do halo e a malha cartesiana, denominado *anisotropia posicional*, que decorre do fato de a grade CIC não ser perfeitamente isotrópica. Para caracterizar e quantificar estes erros, o código implementa a função `mapear_erro`, que realiza um estudo sistemático de translações aleatórias do halo em relação à grade, permitindo separar o viés médio da variância devido à anisotropia.

3.3.1 Definição do erro relativo

Para uma dada concentração c e resolução N , define-se o erro relativo local do potencial numérico em relação ao potencial analítico como

$$\epsilon(\mathbf{r}) = \frac{\Phi_{\text{num}}(\mathbf{r}) - \Phi_{\text{ana}}(\mathbf{r})}{|\Phi_{\text{ana}}(\mathbf{r})|},$$

válido para regiões onde $\Phi_{\text{ana}}(\mathbf{r}) \neq 0$. O sinal de ϵ indica se o potencial numérico é superestimado ($\epsilon > 0$) ou subestimado ($\epsilon < 0$) em relação ao valor exato.

3.3.2 Metodologia de translações aleatórias

Para amostrar a anisotropia posicional, a função `mapear_erro` executa N_{tr} realizações independentes, onde em cada realização o centro do halo é deslocado por um vetor aleatório $\mathbf{d}$

uniformemente distribuído no intervalo

$$d_x, d_y, d_z \in [-0.4 \Delta x, +0.4 \Delta x],$$

com $\Delta x = L/N$ sendo o lado da célula da grade. Este intervalo de deslocamento garante que o centro nunca se afaste mais do que meio voxel da posição original, evitando que o halo se mova para uma célula vizinha, mas sendo suficientemente amplo para explorar diferentes fases da grade.

Para cada translação, o código executa as seguintes operações:

1. Calcula o potencial numérico $\Phi_{\text{num}}(\mathbf{r})$ via a função `phi_num`, utilizando a grade de densidade depositada com o centro deslocado por `d`;
2. Calcula o potencial analítico exato $\Phi_{\text{ana}}(\mathbf{r})$ utilizando o método `potencial` da classe `NFWAnalitico`, avaliado nas mesmas coordenadas da grade;
3. Calcula o campo de erro relativo $\epsilon(\mathbf{r})$ em cada voxel;
4. Realiza um binning radial⁴ para extrair o erro médio em cada casca esférica.

O binning radial é definido a partir do raio mínimo resolvível, $r_{\text{min}} = 0.5 \Delta x$, até o raio virial $r = 1$. Os limites dos bins são logaritmicamente espaçados, com 36 bins no total:

$$r_{\text{bins}} = \log_{10}^{-1}(\log_{10}(r_{\text{min}}) \text{ a } 0, \text{ 36 passos}).$$

Os centros de cada bin são definidos como a média geométrica (ou aritmética no espaço logarítmico) dos limites adjacentes:

$$r_{\text{cent}}^{(j)} = \frac{r_{\text{bins}}^{(j)} + r_{\text{bins}}^{(j+1)}}{2}.$$

Para cada translação i e cada bin radial j , o erro médio naquela casca é calculado como

$$\epsilon_{ij} = \langle \epsilon(\mathbf{r}) \rangle_{r \in [r_j, r_{j+1})},$$

onde a média é tomada sobre todos os voxels cuja distância radial ao centro do halo (após o deslocamento) esteja contida no intervalo do bin. O resultado é armazenado na matriz `erros`, de dimensões $(N_{\text{tr}}, 36)$.

3.3.3 Estatísticas agregadas do erro

Após a execução das N_{tr} translações, a função calcula as seguintes estatísticas para cada bin radial:

4

1. O erro médio (viés sistemático):

$$\bar{\epsilon}(r) = \frac{1}{N_{\text{tr}}} \sum_{i=1}^{N_{\text{tr}}} \epsilon_{ij},$$

que quantifica a tendência média do método numérico de superestimar ou subestimar o potencial.

2. O desvio padrão entre translações:

$$\sigma_{\epsilon}(r) = \sqrt{\frac{1}{N_{\text{tr}}} \sum_{i=1}^{N_{\text{tr}}} (\epsilon_{ij} - \bar{\epsilon}(r))^2},$$

que quantifica a amplitude da anisotropia posicional: quanto maior σ_{ϵ} , maior a dependência do erro com a fase da grade.

3. O erro absoluto máximo:

$$\epsilon_{\text{max}}(r) = \max_i |\epsilon_{ij}|,$$

que fornece o pior cenário entre todas as translações.

4. O percentil 95 do erro absoluto:

$$\epsilon_{95}(r) = \text{Percentil}_{95} \left(\{|\epsilon_{ij}|\}_{i=1}^{N_{\text{tr}}} \right),$$

que indica que 95% das translações apresentam erro absoluto inferior a este valor.

5. O intervalo de confiança de 95% para a média do erro, obtido via bootstrap não-paramétrico:

$$[\epsilon_{\text{IC lo}}(r), \epsilon_{\text{IC hi}}(r)],$$

que quantifica a incerteza estatística na estimativa do viés médio.

A implementação do intervalo de confiança bootstrap utiliza a função `bootstrap_ic`, que opera sobre os N_{tr} valores de ϵ_{ij} para cada bin radial, com $N_{\text{boot}} = 1000$ reamostragens e nível de confiança de 95%.

3.3.4 Implementação computacional

A função `mapear_erro` recebe como parâmetros a concentração c , a resolução N , o número de translações N_{tr} , o número de subvoxels M_{sub} e a semente do gerador aleatório. Internamente, ela instancia o perfil NFW correspondente, define os bins radiais, aloca a matriz `erros` e executa o loop de translações. Ao final, retorna um dicionário contendo todas as métricas calculadas, além dos bins e dos erros individuais para análises posteriores.


```

1 def mapear_erro(c, N=128, Ntr=100, M_sub=8, seed=RNG_SEED):
2     rng = np.random.default_rng(seed)
3     nfw = NFWAnalitico(c)
4     dx = CAIXA_L / N
5     r_min = 0.5 * dx
6     r_bins = np.logspace(np.log10(r_min), 0.0, 36)
7     r_cent = 0.5 * (r_bins[:-1] + r_bins[1:])
8     erros = np.full((Ntr, len(r_cent)), np.nan)
9
10    print(f"\n [mapa_erro] c={c:2d} | N={N} | {Ntr} transl. | M_sub={M_sub}
11    ", flush=True)
12    for i in range(Ntr):
13        desl = rng.uniform(-0.4*dx, 0.4*dx, size=3)
14        print(f" {i+1:3d}/{Ntr}", end="\r", flush=True)
15        Pn, rg = phi_num(nfw, N, desl, M_sub)
16        Pa = nfw.potencial(rg.ravel()).reshape(rg.shape)
17        with np.errstate(invalid="ignore", divide="ignore"):
18            denom = np.abs(Pa)
19            eps = np.where(denom > 1e-10, (Pn-Pa)/denom, np.nan)
20            eps = np.where(np.isfinite(eps), eps, np.nan)
21        rf, ef = rg.ravel(), eps.ravel()
22        for j, (r0, r1) in enumerate(zip(r_bins[:-1], r_bins[1:])):
23            m = (rf >= r0) & (rf < r1)
24            if m.any(): erros[i, j] = np.nanmean(ef[m])
25        del Pn, rg, Pa, eps; gc.collect()
26
27    print()
28    erro_medio = np.nanmean(erros, axis=0)
29    erro_std = np.nanstd(erros, axis=0)
30    erro_abs_max = np.nanmax(np.abs(erros), axis=0)
31    erro_abs_p95 = np.nanpercentile(np.abs(erros), 95, axis=0)
32    ic_lo = np.full(len(r_cent), np.nan)
33    ic_hi = np.full(len(r_cent), np.nan)
34    for j in range(len(r_cent)):
35        ic_lo[j], ic_hi[j] = bootstrap_ic(erros[:, j], n_boot=1000, seed=
36        seed+j)
37
38    return dict(c=c, N=N, r_bins=r_bins, r_cent=r_cent,
39               erro_medio=erro_medio,
40               erro_std =erro_std,
41               erro_max =erro_abs_max,
42               erro_p95 =erro_abs_p95,
43               erro_ic95_lo=ic_lo, erro_ic95_hi=ic_hi,
44               erros_acum=erros)

```

Listing 6: Função mapear_erro para mapeamento do erro espectral e anisotropia

A função é executada para cada concentração do conjunto $\text{CONCS_MAP} = [5, 10, 15, 20, 30]$, gerando um mapa de erro completo que cobre a faixa de concentrações de interesse. Este mapa de erro é a base para a construção do corretor PCHIP descrito na Etapa 4, onde o viés sistemático será interpolado em função do raio e da concentração para permitir a correção de potenciais numéricos em qualquer ponto do espaço de parâmetros.

3.4 Etapa 4 – Função de transferência e correção PCHIP

A etapa anterior estabeleceu o mapa de erro $\epsilon(r, c)$ para as concentrações calibradas $c \in \{5, 10, 15, 20, 30\}$. O passo seguinte consiste em construir uma função de transferência contínua que, dado um potencial numérico Φ_{num} obtido para uma concentração c qualquer (não necessariamente calibrada) e em um raio arbitrário r , produza uma estimativa corrigida Φ_{corr} que se aproxime o máximo possível da solução analítica exata Φ_{ana} . Para tanto, o código implementa a classe `CorretorPCHIP`, que realiza uma interpolação bidimensional do viés via *monotonic cubic Hermite interpolating polynomial* (PCHIP) no raio e interpolação linear na concentração.

3.4.1 Formulação algébrica da correção

Seja o viés relativo definido por

$$\epsilon(r, c) = \frac{\Phi_{\text{num}}(r, c) - \Phi_{\text{ana}}(r, c)}{|\Phi_{\text{ana}}(r, c)|}.$$

Como o potencial gravitacional de halos de matéria escura é sempre negativo ($\Phi_{\text{ana}} < 0$), tem-se $|\Phi_{\text{ana}}| = -\Phi_{\text{ana}}$. Substituindo na definição:

$$\epsilon = \frac{\Phi_{\text{num}} - \Phi_{\text{ana}}}{-\Phi_{\text{ana}}} = \frac{\Phi_{\text{ana}} - \Phi_{\text{num}}}{\Phi_{\text{ana}}} = 1 - \frac{\Phi_{\text{num}}}{\Phi_{\text{ana}}}.$$

Rearranjando a expressão, obtém-se a relação algébrica exata:

$$\frac{\Phi_{\text{num}}}{\Phi_{\text{ana}}} = 1 - \epsilon \implies \Phi_{\text{ana}} = \frac{\Phi_{\text{num}}}{1 - \epsilon}.$$

Portanto, se conhecermos uma estimativa contínua $\hat{\epsilon}(r, c)$ do viés, podemos corrigir o potencial numérico por uma simples divisão escalar:

$$\Phi_{\text{corr}}(r, c) = \frac{\Phi_{\text{num}}(r, c)}{1 - \hat{\epsilon}(r, c)}.$$

A classe `CorretorPCHIP` utiliza exatamente esta relação, com a inclusão de um mecanismo de segurança para evitar instabilidades numéricas: o denominador é restrito ao intervalo $[0.05, 2.0]$ através da função `np.clip`. Este procedimento impede que o denominador se anule

($\epsilon = 1$) ou se torne excessivamente próximo de zero, o que amplificaria o ruído numérico de forma descontrolada.

3.4.2 Interpolação PCHIP no raio

Para cada concentração calibrada c , o método construtor `__init__` da classe `CorretorPCHIP` extrai o erro médio $\bar{\epsilon}(r)$ obtido no mapeamento da Etapa 3. Este erro é armazenado no vetor `r_cent` (centros dos bins radiais) e no vetor `erro_medio` (valores de $\bar{\epsilon}$).

Antes de construir o interpolador, o código realiza um pré-processamento robusto:

1. Identifica os valores finitos de erro através de `np.isfinite`;
2. Se houver pelo menos 4 pontos finitos, os valores não finitos (possíveis NaN oriundos de divisões por zero no centro exato) são preenchidos por interpolação linear (`np.interp`) a partir dos pontos válidos;
3. Caso contrário, todo o vetor de erro é definido como zero.

Em seguida, o domínio radial é estendido para garantir que o interpolador nunca seja solicitado a extrapolar para fora do intervalo coberto pelos dados. O vetor radial é estendido para:

$$r_{\text{ext}} = [0.0, \mathbf{r}_{\text{cent}}, 1.5, 6.0],$$

enquanto o vetor de erro correspondente é:

$$\epsilon_{\text{ext}} = [\epsilon(0), \bar{\epsilon}, 0.0, 0.0].$$

A extensão para $r = 0$ utiliza o valor do erro no primeiro bin radial, assumindo continuidade até o centro. A extensão para $r = 1.5$ e $r = 6.0$ força o erro a se anular fora do domínio da grade, o que é fisicamente razoável, uma vez que o erro numérico tende a zero em regiões onde a densidade é desprezível.

Com os vetores estendidos, o código instancia um interpolador PCHIP da biblioteca SciPy para cada concentração calibrada. A propriedade de monotonicidade do PCHIP é essencial neste contexto, pois evita oscilações espúrias (fenômeno de Runge) que surgiriam com splines cúbicas naturais, especialmente na região da cúspide onde a variação do erro é abrupta.

3.4.3 Interpolação na concentração

Para uma concentração c não pertencente ao conjunto calibrado, o método `fator` realiza uma interpolação linear entre as concentrações adjacentes. Seja $\mathcal{C} = [c_0, c_1, \dots, c_{K-1}]$ o conjunto ordenado de concentrações calibradas, com $c_0 = 5$ e $c_{K-1} = 30$. Para uma dada concentração c :

Esta abordagem assume que o viés varia suavemente com a concentração, o que é consistente com os resultados do mapeamento de erro da etapa anterior.

3.4.4 Aplicação da correção

A correção propriamente dita é realizada pelo método `corrigir`, que recebe o campo de potencial numérico Φ_{num} (um array tridimensional da grade), a grade de distâncias radiais r_{grid} e a concentração c . O método:

1. Calcula $\hat{e}(r_{\text{grid}}, c)$ para todos os voxels simultaneamente através do método `fator`;
2. Calcula o denominador $D = \text{clip}(1 - \hat{e}, 0.05, 2.0)$;
3. Retorna $\Phi_{\text{corr}} = \Phi_{\text{num}}/D$.

Adicionalmente, o método `score_corrigido` aplica a correção especificamente no raio R_{eval} e retorna o score de Nadler corrigido, simplificando a integração com o critério de classificação estelar.

A implementação completa da classe é apresentada a seguir.

```
1 class CorretorPCHIP:
2     def __init__(self, resultados):
3         self.concs = sorted(d["c"] for d in resultados)
4         self._pchip = {}
5         for d in resultados:
6             c = d["c"]
7             r = d["r_cent"].copy()
8             e = d["erro_medio"].copy()
9             ok = np.isfinite(e)
10            if ok.sum() >= 4:
11                e = np.where(ok, e, np.interp(r, r[ok], e[ok]))
12            else:
13                e = np.zeros_like(e)
14                r_ext = np.concatenate([[0.0], r, [1.5, 6.0]])
15                e_ext = np.concatenate([[e[0]], e, [0.0, 0.0]])
16                self._pchip[c] = PchipInterpolator(r_ext, e_ext, extrapolate=
False)
17            print(f" [CorretorPCHIP] c = {self.concs}")
18
19     def fator(self, r, c):
20         r = np.asarray(r, dtype=float)
21         c_arr = np.array(self.concs, dtype=float)
22         def _ev(cv):
23             v = self._pchip[cv](r)
24             return np.where(np.isfinite(v), v, 0.0)
25         if c <= c_arr[0]: return _ev(c_arr[0])
26         if c >= c_arr[-1]: return _ev(c_arr[-1])
27         idx = int(np.searchsorted(c_arr, c)) - 1
28         c0, c1 = c_arr[idx], c_arr[idx+1]
29         t = (c - c0) / (c1 - c0)
```

```

30     return (1-t)*_ev(c0) + t*_ev(c1)
31
32     def corrigir(self, Pn, rg, c):
33         eps = self.fator(rg, c)
34         denom = np.clip(1.0 - eps, 0.05, 2.0)
35         return Pn / denom
36
37     def score_corrigido(self, phi_n_val, r_val, c):
38         eps = float(self.fator(np.array([r_val]), c)[0])
39         denom = float(np.clip(1.0 - eps, 0.05, 2.0))
40         return score_phi(phi_n_val / denom, PHI_REF)

```

Listing 7: Classe `CorretorPCHIP` para correção do viés gravitacional

A função de transferência PCHIP construída nesta etapa é o núcleo do método de correção proposto. Ela será aplicada nos halos sintéticos gerados na simulação Monte Carlo da Etapa 6, permitindo a comparação direta entre as classificações estelares obtidas com o potencial numérico bruto, o potencial corrigido e a solução analítica exata.

3.5 Etapa 5 – Validação cruzada

A etapa anterior estabeleceu a função de transferência PCHIP que permite corrigir o potencial numérico obtido com resolução N_{cal} para que ele se aproxime do potencial analítico exato. Antes de aplicar a correção em produção, é imperativo verificar se o corretor realmente recupera a acurácia de uma simulação de alta resolução. Para tanto, o código implementa a função `validacao_cruzada`, que compara quatro perfis de potencial para uma mesma concentração c_{teste} :

1. O potencial analítico exato (*ground truth*);
2. O potencial numérico de alta resolução $N_{\text{ref}} = 128$, que serve como referência numérica confiável;
3. O potencial numérico de baixa resolução $N_{\text{cal}} = 64$ sem correção;
4. O potencial numérico de baixa resolução $N_{\text{cal}} = 64$ após a aplicação do corretor PCHIP.

O objetivo deste teste é demonstrar que $N_{\text{cal}} + \text{PCHIP} \approx N_{\text{ref}} \approx \text{analítico}$, validando assim a eficácia da correção e justificando o uso de resoluções mais baixas (e, portanto, mais rápidas) nas simulações de produção.

3.5.1 Metodologia de comparação

A função `validacao_cruzada` recebe como argumentos uma instância do `CorretorPCHIP` já calibrado, a concentração de teste c_{teste} (padrão 10.0), as resoluções N_{ref} e N_{cal} , o número de subvoxels M_{sub} e uma semente aleatória.

Inicialmente, a função gera um deslocamento aleatório d uniformemente distribuído no intervalo

$$d_x, d_y, d_z \in [-0.05, +0.05],$$

com o único propósito de quebrar a simetria da grade em relação ao centro do halo, evitando que os resultados sejam dependentes de uma configuração particular da malha cartesiana.

Em seguida, são calculados:

1. O potencial analítico $\Phi_{\text{ana}}(r)$ para uma grade radial fina de 400 pontos logaritmicamente espaçados entre $10^{-2.5}$ e 1.0;
2. O potencial numérico de referência Φ_{ref} utilizando a função `phi_num` com resolução N_{ref} ;
3. O potencial numérico de baixa resolução Φ_{cal} utilizando `phi_num` com resolução N_{cal} ;
4. O potencial corrigido $\Phi_{\text{corr}} = \text{corretor.corrigir}(\Phi_{\text{cal}}, r_{\text{grid}}, c_{\text{teste}})$.

3.5.2 Binning radial

Para comparar os quatro campos de potencial em uma base comum, a função realiza um binning radial sobre a grade de baixa resolução. O espaçamento da grade grossa é

$$\Delta x_{\text{cal}} = \frac{L}{N_{\text{cal}}}.$$

Os limites dos bins são definidos logaritmicamente a partir do raio mínimo resolvível pela grade grossa, $r_{\text{min}} = 0.5 \Delta x_{\text{cal}}$, até o raio virial $r = 1$, com 28 bins:

$$r_{\text{bins}} = \log_{10}^{-1}(\log_{10}(r_{\text{min}}) \text{ a } 0, 28 \text{ passos}).$$

Os centros dos bins são

$$r_{\text{cent}}^{(j)} = \frac{r_{\text{bins}}^{(j)} + r_{\text{bins}}^{(j+1)}}{2}.$$

Para cada um dos campos de potencial (Φ_{ref} , Φ_{cal} , Φ_{corr}), a função calcula o valor médio dentro de cada bin radial, produzindo perfis radiais binned que podem ser diretamente comparados entre si e com o perfil analítico $\Phi_{\text{ana}}(r_{\text{cent}})$. Este procedimento é realizado pela função interna `_bin`:

```

1 def _bin(campo, grade):
2     rf, pf = grade.ravel(), campo.ravel()
3     med = np.full(len(r_cent), np.nan)
4     for j, (r0, r1) in enumerate(zip(r_bins[:-1], r_bins[1:])):
5         m = (rf >= r0) & (rf < r1)
6         if m.any(): med[j] = np.mean(pf[m])
7     return med

```

3.5.3 Métricas de erro

Para quantificar a eficácia da correção, a função calcula o erro relativo percentual em cada bin radial, antes e depois da correção:

$$\varepsilon_{\text{antes}}(r) = \left| \frac{\Phi_{\text{cal}}(r) - \Phi_{\text{ana}}(r)}{\Phi_{\text{ana}}(r)} \right| \times 100,$$

$$\varepsilon_{\text{depois}}(r) = \left| \frac{\Phi_{\text{corr}}(r) - \Phi_{\text{ana}}(r)}{\Phi_{\text{ana}}(r)} \right| \times 100.$$

A partir destes vetores de erro, a função extrai as seguintes métricas agregadas:

A escolha de reportar o erro máximo e o percentil 95, e não apenas a média sobre todos os raios, é motivada pela observação de que uma média integrada pode mascarar um erro residual elevado perto da cúspide central, precisamente onde o critério de Nadler é mais sensível. Esta preocupação está explicitamente documentada no código (*nota de avaliação, Seção 4.1*).

3.5.4 Implementação computacional

A função `validacao_cruzada` orquestra todas estas operações, retornando um dicionário contendo os perfis radiais, as métricas de erro e os parâmetros do teste.

```
1 def validacao_cruzada(corretor, c_teste=10.0, N_ref=128, N_cal=64,
2                       M_sub=8, seed=123):
3     rng = np.random.default_rng(seed)
4     nfw = NFWAnalitico(c_teste)
5     desl = rng.uniform(-0.05, 0.05, size=3)
6     print(f"\n [val_cruzada] c={c_teste} | N_ref={N_ref} | N_cal={N_cal}")
7
8     r_ana = np.logspace(-2.5, 0.0, 400)
9     phi_a = nfw.potencial(r_ana)
10
11     Pr, rr = phi_num(nfw, N_ref, desl, M_sub)
12     print(f"      Grade calibrada N={N_cal}:")
13     Pc, rc = phi_num(nfw, N_cal, desl, M_sub)
14     Pcc = corretor.corrigir(Pc, rc, c_teste)
15
16     dx_c = CAIXA_L / N_cal
17     r_bins = np.logspace(np.log10(0.5*dx_c), 0.0, 28)
18     r_cent = 0.5 * (r_bins[:-1] + r_bins[1:])
19
20     def _bin(campo, grade):
21         rf, pf = grade.ravel(), campo.ravel()
22         med = np.full(len(r_cent), np.nan)
23         for j, (r0, r1) in enumerate(zip(r_bins[:-1], r_bins[1:])):
24             m = (rf>=r0)&(rf<r1)
25             if m.any(): med[j] = np.mean(pf[m])
```

```

26     return med
27
28     perfil_ana_b = nfw.potencial(r_cent)
29     perfil_cal   = _bin(Pc, rc)
30     perfil_corr  = _bin(Pcc, rc)
31
32     with np.errstate(invalid="ignore", divide="ignore"):
33         msk = np.abs(perfil_ana_b) > 1e-10
34         erro_pt_ant = np.where(msk, np.abs(perfil_cal - perfil_ana_b) /
35                                 np.abs(perfil_ana_b) * 100, np.nan)
36         erro_pt_dep = np.where(msk, np.abs(perfil_corr - perfil_ana_b) /
37                                 np.abs(perfil_ana_b) * 100, np.nan)
38
39     i_reval = int(np.argmin(np.abs(r_cent - R_EVAL)))
40
41     res = dict(r_ana=r_ana, phi_ana=phi_a, r_cent=r_cent,
42               perfil_ref =_bin(Pr, rr),
43               perfil_cal =perfil_cal,
44               perfil_corr =perfil_corr,
45               perfil_ana_b=perfil_ana_b,
46               erro_pt_ant=erro_pt_ant, erro_pt_dep=erro_pt_dep,
47               erro_max_ant=float(np.nanmax(erro_pt_ant)),
48               erro_max_dep=float(np.nanmax(erro_pt_dep)),
49               erro_p95_ant=float(np.nanpercentile(erro_pt_ant, 95)),
50               erro_p95_dep=float(np.nanpercentile(erro_pt_dep, 95)),
51               erro_reval_ant=float(erro_pt_ant[i_reval]),
52               erro_reval_dep=float(erro_pt_dep[i_reval]),
53               c=c_teste, N_ref=N_ref, N_cal=N_cal)
54     del Pr, rr, Pc, Pcc, rc; gc.collect()
55     return res

```

Listing 8: Função `validacao_cruzada` para validação cruzada da correção PCHIP

Os critérios de validação são os seguintes:

- A correção deve reduzir o erro máximo ($\varepsilon_{\max}$) e o percentil 95 (ε_{95}) em pelo menos uma ordem de grandeza;
- O erro no raio de avaliação $\varepsilon(R_{\text{eval}})$ deve cair para valores inferiores a 1%, que é a meta de precisão para a aplicação do critério de Nadler;
- O perfil corrigido $\Phi_{\text{corr}}(r)$ deve sobrepor-se visualmente ao perfil de referência N_{ref} e ao analítico em toda a extensão radial.

A validação cruzada é aplicada para a concentração $c_{\text{teste}} = 10$, que está no centro da faixa calibrada e é a concentração utilizada como referência para o potencial Φ_{ref} no critério de Nadler. A aprovação neste teste é condição necessária para que o corretor seja considerado apto para a produção de resultados científicos.

3.6 Etapa 6 – Teste de sensibilidade ao perfil (Hernquist)

As etapas anteriores estabeleceram o corretor PCHIP utilizando exclusivamente o perfil NFW como referência para a calibração. Para avaliar se o corretor é específico ao NFW ou se possui caráter mais geral, o código implementa o teste de sensibilidade `teste_sensibilidade_hernquist` que aplica o mesmo corretor calibrado em NFW ao perfil de Hernquist [?] e quantifica a eficácia da correção.

A motivação científica para este teste é fundamentada na semelhança das cúspides centrais: tanto o NFW quanto o Hernquist possuem comportamento assintótico $\rho \propto r^{-1}$ para $r \rightarrow 0$. O viés numérico introduzido pela deposição CIC e pela FFT origina-se predominantemente da singularidade da cúspide, e não da forma global do perfil em raios maiores. Portanto, espera-se que um corretor calibrado em NFW também seja eficaz na redução do erro para o perfil de Hernquist. O teste quantifica a transferabilidade do método e demonstra sua robustez.

3.6.1 Metodologia

A função `teste_sensibilidade_hernquist` recebe como argumentos uma instância do `CorretorPCHIP` já calibrado, uma concentração equivalente c_{equiv} (padrão 10.0), a resolução N da grade (padrão 64), o número de subvoxels M_{sub} (padrão 4) e uma semente aleatória.

O procedimento é o seguinte:

1. Instancia o perfil de Hernquist com a concentração equivalente c_{equiv} , utilizando a classe `HernquistAnalitico`;
2. Gera um deslocamento aleatório $\mathbf{d}$ uniformemente distribuído no intervalo

$$d_x, d_y, d_z \in [-0.05, +0.05],$$

para quebrar a simetria da grade;

3. Calcula o potencial numérico Φ_{num} para o perfil de Hernquist utilizando a função `phi_num`;
4. Aplica o corretor PCHIP (calibrado em NFW) ao potencial numérico, utilizando o mesmo c_{equiv} como argumento para o interpolador:

$$\Phi_{\text{corr}} = \text{corretor.corrigir}(\Phi_{\text{num}}, r_{\text{grid}}, c_{\text{equiv}});$$

5. Calcula o potencial analítico exato do Hernquist Φ_{ana} para os mesmos pontos da grade;
6. Realiza um binning radial com 28 bins logaritmicamente espaçados, a partir de $r_{\text{min}} = 0.5 \Delta x$ até $r = 1$, e extrai os perfis radiais binned para os três campos ($\Phi_{\text{ana}}, \Phi_{\text{num}}, \Phi_{\text{corr}}$).

A função interna `_bin` realiza o mesmo procedimento de médias radiais descrito na Etapa 5, garantindo consistência metodológica.

3.6.2 Métricas de eficácia

A eficácia da correção é quantificada através do erro relativo absoluto médio (em porcentagem), definido para cada método como

$$\varepsilon_{\text{num}} = \left\langle \left| \frac{\Phi_{\text{num}} - \Phi_{\text{ana}}}{\Phi_{\text{ana}}} \right| \right\rangle \times 100,$$

$$\varepsilon_{\text{corr}} = \left\langle \left| \frac{\Phi_{\text{corr}} - \Phi_{\text{ana}}}{\Phi_{\text{ana}}} \right| \right\rangle \times 100,$$

onde a média é tomada sobre todos os bins radiais.

A eficiência da correção é definida como a redução percentual do erro médio:

$$\eta = \left(1 - \frac{\varepsilon_{\text{corr}}}{\varepsilon_{\text{num}}} \right) \times 100\%.$$

A interpretação é direta: $\eta \approx 100\%$ indica que a correção eliminou praticamente todo o viés; $\eta \approx 0\%$ indica que a correção não surtiu efeito; valores negativos indicariam que a correção piorou o resultado, o que não é esperado.

3.6.3 Implementação computacional

```

1 def teste_sensibilidade_hernquist(corretor, c_equiv=10.0, N=64,
2                               M_sub=4, seed=77):
3     rng = np.random.default_rng(seed)
4     hq = HernquistAnalitico(c_equiv)
5     desl = rng.uniform(-0.05, 0.05, size=3)
6
7     print(f"\n [Hernquist] c_equiv={c_equiv} | N={N}")
8     Pn, rg = phi_num(hq, N, desl, M_sub)
9     Pc      = corretor.corrigir(Pn, rg, c_equiv)
10
11     dx      = CAIXA_L / N
12     r_bins = np.logspace(np.log10(0.5*dx), 0.0, 28)
13     r_cent = 0.5 * (r_bins[:-1] + r_bins[1:])
14
15     def _bin(campo, grade):
16         rf, pf = grade.ravel(), campo.ravel()
17         med = np.full(len(r_cent), np.nan)
18         for j, (r0, r1) in enumerate(zip(r_bins[:-1], r_bins[1:])):
19             m = (rf>=r0)&(rf<r1)
20             if m.any(): med[j] = np.mean(pf[m])
21     return med

```

```

22
23     pa_b = hq.potencial(r_cent)
24     pn_b = _bin(Pn, rg)
25     pc_b = _bin(Pc, rg)
26
27     r_ana = np.logspace(-2.5, 0.0, 400)
28     phi_ha = hq.potencial(r_ana)
29
30     with np.errstate(invalid="ignore", divide="ignore"):
31         msk = np.abs(pa_b) > 1e-10
32         ea = np.where(msk, (pn_b - pa_b)/np.abs(pa_b)*100, np.nan)
33         ed = np.where(msk, (pc_b - pa_b)/np.abs(pa_b)*100, np.nan)
34
35     e_ant = float(np.nanmean(np.abs(ea)))
36     e_dep = float(np.nanmean(np.abs(ed)))
37     efic = (1 - e_dep/max(e_ant, 1e-10)) * 100
38
39     del Pn, Pc, rg; gc.collect()
40     return dict(hq=hq, r_ana=r_ana, phi_ha=phi_ha,
41                r_cent=r_cent, pa_b=pa_b, pn_b=pn_b, pc_b=pc_b,
42                ea=ea, ed=ed, e_ant=e_ant, e_dep=e_dep, efic=efic,
43                c_equiv=c_equiv, N=N)

```

Listing 9: Função teste_sensibilidade_hernquist para validação da transferabilidade do corretor

3.6.4 Resultados esperados e interpretação

A expectativa física é que $\varepsilon_{\text{corr}} \ll \varepsilon_{\text{num}}$, com eficiência $\eta > 80\%$. Valores elevados de η indicam que o viés numérico é efetivamente dominado pela cúspide central, comum a ambos os perfis, e que o corretor PCHIP é robusto o suficiente para ser aplicado a diferentes modelos analíticos. A demonstração de transferabilidade fortalece a tese de que o corretor captura o erro intrínseco da discretização CIC-FFT, independentemente da forma funcional do perfil de densidade em raios externos.

Este teste complementa a validação cruzada da Etapa 5: enquanto aquela verifica a consistência numérica entre resoluções de um mesmo perfil, esta verifica a generalidade física do corretor entre perfis distintos. Ambos são requisitos para que o método possa ser aplicado com confiança em cenários onde o perfil real dos halos pode desviar-se do NFW.

3.7 Etapa 6.5 – Estudos adicionais de robustez

Além da validação cruzada (Etapa 5) e do teste de sensibilidade ao perfil (Etapa 6), o código implementa quatro estudos adicionais de robustez que investigam a dependência dos resultados numéricos em relação a parâmetros fundamentais do método: (i) o número de subvoxels por

célula (M_{sub}), (ii) o tamanho da caixa periódica (L), (iii) a escolha da concentração de referência para o potencial Φ_{ref} , e (iv) a escalabilidade computacional do método com a resolução da grade. Estes estudos atendem diretamente às exigências de uma análise numérica rigorosa, documentando a convergência e as fontes de incerteza sistemática.

3.7.1 Convergência do número de subvoxels CIC (M_{sub})

A função `estudo_convergencia_Msub` investiga a convergência do erro do potencial numérico em função do número de subvoxels por eixo utilizado na deposição CIC. O estudo é realizado para uma concentração fixa $c = 10$, resolução $N = 32$ e valores de $M_{\text{sub}} \in \{2, 4, 8, 16\}$.

Em cada execução, o centro do halo é deslocado por um vetor fixo (para isolar o efeito de M_{sub} das flutuações de fase da grade) definido como

$$\mathbf{d} = \mathbf{u} \cdot \frac{L}{N}, \quad \mathbf{u} \in [-0.3, +0.3]^3,$$

onde $\mathbf{u}$ é gerado aleatoriamente com semente fixa, garantindo reprodutibilidade. O código então calcula o potencial numérico Φ_{num} para cada M_{sub} e o compara com o potencial analítico Φ_{ana} para extrair duas métricas de erro:

O tempo de CPU de cada execução também é registrado, permitindo avaliar o custo-benefício de aumentar M_{sub} . A função retorna um dicionário com os erros e tempos para cada valor testado.

```

1 def estudo_convergencia_Msub(c=10.0, N=32, M_subs=(2, 4, 8, 16), seed=999):
2     rng = np.random.default_rng(seed)
3     nfw = NFWAnalitico(c)
4     desl = rng.uniform(-0.3, 0.3, size=3) * (CAIXA_L / N)
5     dx = CAIXA_L / N
6
7     erros_c, erros_e, tempos = [], [], []
8     for Msub in M_subs:
9         t0 = time.perf_counter()
10        Pn, rg = phi_num(nfw, N, desl, Msub)
11        dt = time.perf_counter() - t0
12        Pa = nfw.potencial(rg.ravel()).reshape(rg.shape)
13        with np.errstate(invalid="ignore", divide="ignore"):
14            eps = np.abs((Pn - Pa) / np.abs(Pa)) * 100
15            i_c = np.unravel_index(np.argmin(rg), rg.shape)
16            m_ev = (rg > 0.9*R_EVAL) & (rg < 1.1*R_EVAL)
17            erros_c.append(float(eps[i_c]))
18            erros_e.append(float(np.nanmean(eps[m_ev])) if m_ev.any() else np.
19                nan)
20            tempos.append(dt)
21        del Pn, rg, Pa, eps; gc.collect()

```

```

21     return dict(M_subs=list(M_subs), erro_central=erros_c,
22                 erro_reval=erros_e, tempos=tempos, N=N, c=c)

```

Listing 10: Função estudo_convergencia_Msub

3.7.2 Sensibilidade ao tamanho da caixa periódica (L)

A função estudo_sensibilidade_Lbox avalia o efeito do tamanho da caixa periódica L sobre o erro do potencial no raio R_{eval} . O estudo é motivado pela possibilidade de que halos próximos às bordas da caixa sofram auto-interação periódica espúria devido à periodicidade inerente da FFT.

O código mantém fixos $N = 32$, $M_{\text{sub}} = 4$ e a concentração $c = 10$, e varia o tamanho da caixa para

$$L \in \{2.0, 3.0, 4.0\}.$$

Para cada valor de L , a constante global CAIXA_L é temporariamente modificada, com o valor original restaurado ao final (independentemente de exceções) através de um bloco try/finally. O deslocamento do halo é gerado aleatoriamente como

$$\mathbf{d} \in [-0.3 \Delta x, +0.3 \Delta x]^3, \quad \Delta x = L/N,$$

garantindo que a amplitude do deslocamento em relação ao tamanho da célula seja consistente entre os diferentes valores de L .

A métrica de erro é o erro relativo médio na casca em torno de R_{eval} :

$$\varepsilon_L(R_{\text{eval}}) = \left\langle \left| \frac{\Phi_{\text{num}} - \Phi_{\text{ana}}}{\Phi_{\text{ana}}} \right| \right\rangle_{0.9R_{\text{eval}} < r < 1.1R_{\text{eval}}} \times 100.$$

```

1  def estudo_sensibilidade_Lbox(c=10.0, N=32, M_sub=4, Ls=(2.0, 3.0, 4.0),
2      seed=999):
3      global CAIXA_L
4      rng = np.random.default_rng(seed)
5      nfw = NFWAnalitico(c)
6      resultados = []
7      L_original = CAIXA_L
8      try:
9          for L in Ls:
10             CAIXA_L = L
11             dx = CAIXA_L / N
12             desl = rng.uniform(-0.3*dx, 0.3*dx, size=3)
13             Pn, rg = phi_num(nfw, N, desl, M_sub)
14             Pa = nfw.potencial(rg.ravel()).reshape(rg.shape)
15             m_ev = (rg > 0.9*R_EVAL) & (rg < 1.1*R_EVAL)
16             with np.errstate(invalid="ignore", divide="ignore"):
17                 eps = np.abs((Pn - Pa) / np.abs(Pa)) * 100

```

```

17         e_ev = float(np.nanmean(eps[m_ev])) if m_ev.any() else np.nan
18         resultados.append(e_ev)
19         del Pn, rg, Pa, eps; gc.collect()
20     finally:
21         CAIXA_L = L_original
22     return dict(Ls=list(Ls), erro_reval=resultados, N=N, c=c)

```

Listing 11: Função estudo_sensibilidade_Lbox

É importante notar que, como implementado, o estudo mantém N fixo enquanto varia L . Consequentemente, o espaçamento da grade $\Delta x = L/N$ também varia, de modo que o resultado reflete uma combinação do efeito da periodicidade com o efeito da resolução espacial.

3.7.3 Sensibilidade à escolha da concentração de referência (Φ_{ref})

O critério de Nadler utiliza um potencial de referência Φ_{ref} calculado para um perfil NFW com concentração $c_{\text{ref}} = 10$. Para avaliar o impacto desta escolha sobre a classificação estelar, a função `sensibilidade_phi_ref` calcula o score de Nadler para um halo de teste com $c_{\text{halo}} = 8.0$ utilizando diferentes concentrações de referência

$$c_{\text{ref}} \in \{5, 10, 15, 20, 30\}.$$

Para cada c_{ref} , calcula-se:

$$\Phi_{\text{ref}}(c_{\text{ref}}) = \Phi_{\text{NFW}}(R_{\text{eval}}; c_{\text{ref}}),$$

e o score correspondente:

$$S(c_{\text{ref}}) = \frac{|\Phi_{\text{halo}}(R_{\text{eval}})|}{|\Phi_{\text{ref}}(c_{\text{ref}})|}.$$

O halo é classificado como estelar se $S(c_{\text{ref}}) > \text{SCORE_LIM} = 0.85$ e $V_{\text{max}} > 12$ km/s. A função retorna uma tabela que mostra como a classificação binária (estelar/escuro) varia com a escolha de c_{ref} .

```

1 def sensibilidade_phi_ref(c_refs=(5.0, 10.0, 15.0, 20.0, 30.0),
2                           c_halo=8.0):
3     nfw_halo = NFWAnalitico(c_halo)
4     phi_halo = float(nfw_halo.potencial(np.array([R_EVAL]))[0])
5     linhas = []
6     for cr in c_refs:
7         phi_ref_cr = float(NFWAnalitico(cr).potencial(np.array([R_EVAL]))
8                             [0])
9         score = abs(phi_halo) / abs(phi_ref_cr)
10        classifica = score > SCORE_LIM
11        linhas.append(dict(c_ref=cr, phi_ref=phi_ref_cr, score=score,
12                           estelar=classifica))
12    return dict(c_halo=c_halo, phi_halo=phi_halo, linhas=linhas)

```

Listing 12: Função sensibilidade_phi_ref

3.7.4 *Projeção de escalabilidade computacional*

Para planejar execuções futuras com resoluções mais altas (ex.: $N = 256$), a função `projecao_escalabilidade` realiza uma projeção empírica do tempo de CPU. Em vez de assumir a escalabilidade teórica $t \propto N^3$, que é otimista pois ignora efeitos de *cache* e *over-head* do Python, a função mede o tempo para três resoluções ($N = 32, 64, 128$) e ajusta uma lei de potência no espaço log-log:

$$\log t(N) = p \log N + b \implies t(N) = e^b N^p.$$

O ajuste é realizado pela função `np.polyfit` com grau 1 sobre os pares $(\log N, \log t)$.

Para uma resolução alvo $N_{\text{proj}} = 256$, a função projeta o tempo por halo e estima o custo total da calibração e da produção. O custo total da calibração é estimado como

$$t_{\text{calibrao}} = t(N) \times N_{\text{trans}} \times |\text{CONCS_MAP}|,$$

onde $N_{\text{trans}} = 100$ é o número de translações por concentração e $|\text{CONCS_MAP}| = 5$. O custo da produção é estimado como

$$t_{\text{produto}} = t(N) \times N_{\text{halos}},$$

com $N_{\text{halos}} = 100$ (o número de halos da amostra de Monte Carlo).

A função também calcula a razão entre a projeção ajustada e a projeção N^3 teórica:

$$R_{\text{terica}} = \left(\frac{N_{\text{proj}}}{N_0} \right)^3, \quad R_{\text{ajustada}} = \left(\frac{N_{\text{proj}}}{N_0} \right)^p,$$

onde N_0 é a menor resolução medida. A comparação entre R_{terica} e R_{ajustada} quantifica o desvio da escalabilidade ideal.

```
1 def projecao_escalabilidade(N_alvo=(32, 64, 128), M_sub=8,
2     N_trans_referencia=100,
3     N_halos_referencia=100, c_amostra=10.0, seed
4     =999,
5     N_projetado=256):
6     rng = np.random.default_rng(seed)
7     nfw = NFWAnalitico(c_amostra)
8     tempos = {}
9     for N in N_alvo:
10         dx = CAIXA_L / N
11         desl = rng.uniform(-0.3*dx, 0.3*dx, size=3)
12         t0 = time.perf_counter()
```

```

11     Pn, rg = phi_num(nfw, N, desl, M_sub)
12     dt = time.perf_counter() - t0
13     tempos[N] = dt
14     del Pn, rg; gc.collect()
15
16     Ns = np.array(sorted(tempos))
17     ts = np.array([tempos[n] for n in Ns])
18     if len(Ns) >= 2:
19         p, b = np.polyfit(np.log(Ns), np.log(ts), 1)
20         t_proj = float(np.exp(b) * N_projetado**p)
21         razao_teorica_N3 = (N_projetado / Ns[0]) ** 3
22         razao_ajustada = (N_projetado / Ns[0]) ** p
23     else:
24         p = b = t_proj = razao_teorica_N3 = razao_ajustada = float("nan")
25
26     proj_min = {}
27     for N in list(Ns) + [N_projetado]:
28         t_N = tempos.get(N, t_proj if N == N_projetado else np.nan)
29         custo_total_s = t_N * N_trans_referencia * len(CONCS_MAP) \
30             + t_N * N_halos_referencia
31         proj_min[int(N)] = custo_total_s / 60.0
32
33     return dict(tempos=tempos, expoente_ajustado=float(p),
34                razao_teorica_N3=razao_teorica_N3,
35                razao_ajustada=razao_ajustada,
36                N_projetado=N_projetado, t_projetado=t_proj,
37                projecao_min=proj_min)

```

Listing 13: Função projecao_escalabilidade

Os resultados destes quatro estudos, juntamente com as validações das etapas anteriores, são consolidados em um relatório de robustez exportado pela função `exportar_csv_robustez`, fornecendo documentação completa da confiabilidade numérica do método.

3.8 Etapa 7 – Simulação de Monte Carlo

Uma vez estabelecida a função de transferência PCHIP e validada sua eficácia por meio dos testes de validação cruzada e sensibilidade ao perfil, a etapa seguinte consiste em aplicar o corretor a uma população sintética de halos representativa da zona de transição crítica do critério de Nadler. Para tanto, o código implementa a função `monte_carlo`, que gera uma amostra de halos de matéria escura com massas e concentrações realistas, calcula seus potenciais numéricos (brutos e corrigidos) e os classifica segundo o critério estelar, permitindo a quantificação da taxa de recuperação de halos erroneamente classificados como escuros pelo método numérico sem correção.

3.8.1 Amostragem de massa e concentração

A amostra é gerada em duas fases: uma amostra-base uniforme e, quando necessário, uma amostragem estratificada complementar.

Amostra-base uniforme. A massa virial dos halos é amostrada uniformemente em escala logarítmica:

$$\log_{10} \left(\frac{M_{\text{vir}}}{M_{\odot}} \right) \sim \mathcal{U}(7.0, 8.5).$$

Esta faixa de massa corresponde a halos com velocidade virial $V_{\text{vir}} \approx 3$ a 15 km/s, cruzando exatamente o limiar de reionização $V_{\text{max}} = 12$ km/s.

A concentração c é amostrada a partir de uma distribuição log-normal com dependência de massa, seguindo a relação aproximada de Dutton & Macciò (2014):

$$\log_{10} c_{\text{med}}(M) = a + b (\log_{10} M - 12),$$

com $a = 0.905$ e $b = -0.101$. A dispersão log-normal é fixada em $\sigma_{\log c} = 0.11$ dex, e a concentração é truncada no intervalo $c \in [5, 35]$. Formalmente:

$$\log_{10} c \sim \mathcal{N}(\log_{10} c_{\text{med}}(M), 0.11^2), \quad c = \text{clip}(10^{\log_{10} c}, 5, 35).$$

Esta relação substitui a distribuição $c \sim \text{LogNormal}(\ln 12, 0.25)$ fixa e sem dependência de massa de versões anteriores, conforme apontado pela avaliação como não fisicamente motivada.

Amostragem estratificada complementar. A amostra-base uniforme contém tipicamente apenas 2 a 5 halos estelares (segundo o critério analítico) em 100 realizações. Esta baixa contagem torna a estimativa da taxa de recuperação estatisticamente frágil. Para contornar este problema sem abandonar a amostragem original, o código implementa uma **amostragem estratificada**:

1. Conta-se o número $n_{\text{est,base}}$ de halos estelares na amostra-base segundo o critério analítico;
2. Se $n_{\text{est,base}} < N_{\text{min_estelar}} = 30$, são sorteados halos adicionais na sub-faixa de massa elevada

$$\log_{10} M \in [8.0, 8.5],$$

onde a probabilidade de obter halos estelares é maior;

3. Os sorteios são realizados um a um até que o número total de halos estelares (analíticos) atinja $N_{\text{min_estelar}}$, ou até que um número máximo de tentativas ($20 \times N_{\text{min_estelar}}$) seja excedido.

Cada halo adicional é identificado pela coluna `estrato` com valor 1 (os halos da amostra-base têm `estrato` = 0), permitindo que o leitor reproduza a análise considerando apenas a amostra uniforme, se assim o desejar. Esta estratificação por desenho é explicitamente documentada na saída do código.

3.8.2 Classificação de cada halo

Para cada halo da amostra (base + estratificada), o código realiza os seguintes passos:

Parâmetros físicos. A massa é convertida para unidades físicas $M = 10^{\log_{10} M}$. A velocidade virial é calculada pela função `Mvir_para_Vvir`:

$$V_{\text{vir}} = \sqrt{\frac{GM}{r_{\text{vir}}}}, \quad r_{\text{vir}} = \left(\frac{3M}{4\pi \Delta_c \rho_{\text{crit}}} \right)^{1/3}.$$

A velocidade máxima física é obtida por

$$V_{\text{max}} = v_{\text{max}}^{\text{adim}}(c) \times V_{\text{vir}},$$

onde $v_{\text{max}}^{\text{adim}}(c)$ é o valor adimensional calculado pelo método `vmax_adim` da classe `NFWAnalitico`.

Classificação analítica (padrão ouro). O potencial analítico no raio de avaliação é

$$\Phi_{\text{ana}} = \Phi_{\text{NFW}}(R_{\text{eval}}; c).$$

O score analítico é

$$S_{\text{ana}} = \frac{|\Phi_{\text{ana}}|}{|\Phi_{\text{ref}}|}.$$

A classificação analítica é

$$\mathcal{C}_{\text{ana}} = (S_{\text{ana}} > 0.85) \wedge (V_{\text{max}} > 12 \text{ km/s}).$$

Classificação numérica e corrigida. Para a classificação numérica, o código gera um deslocamento aleatório d uniformemente distribuído no intervalo

$$d_x, d_y, d_z \in [-0.3 \Delta x, +0.3 \Delta x], \quad \Delta x = L/N,$$

onde $N = N_{\text{grade}}$ é a resolução da grade. O potencial numérico bruto Φ_{num} é obtido via `phi_num` com este deslocamento. O potencial corrigido é

$$\Phi_{\text{corr}} = \frac{\Phi_{\text{num}}}{1 - \hat{\epsilon}(r, c)},$$

com $\hat{\epsilon}$ fornecido pelo `CorretorPCHIP`.

Para extrair o potencial no raio R_{eval} , o código define um raio alvo

$$r_{\text{alvo}} = \max(R_{\text{eval}}, 1.5 \Delta x),$$

e uma margem

$$\text{margem} = \max(1.5 \Delta x, 0.25 r_{\text{alvo}}).$$

A máscara esférica é então

$$\mathcal{M} = \{\mathbf{r} \mid r_{\text{alvo}} - \text{margem} < r < r_{\text{alvo}} + \text{margem}\}.$$

Se $|\mathcal{M}| \geq 8$, o potencial é tomado como a média sobre a máscara:

$$\Phi = \langle \Phi(\mathbf{r}) \rangle_{\mathbf{r} \in \mathcal{M}}.$$

Caso contrário, utiliza-se o valor do voxel mais próximo de r_{alvo} :

$$\Phi = \Phi\left(\arg \min_{\mathbf{r}} |r - r_{\text{alvo}}|\right).$$

Os scores numéricos e corrigidos são então calculados como

$$S_{\text{num}} = \frac{|\Phi_{\text{num}}|}{|\Phi_{\text{ref}}|}, \quad S_{\text{corr}} = \frac{|\Phi_{\text{corr}}|}{|\Phi_{\text{ref}}|},$$

e as classificações $\mathcal{C}_{\text{num}}$ e $\mathcal{C}_{\text{corr}}$ são obtidas pelo mesmo critério dual.

3.8.3 Métricas de desempenho

Ao final do loop sobre todos os halos, a função calcula as seguintes métricas:

- **Falsos escuros:** halos que são estelares no analítico, mas escuros no numérico bruto:

$$\mathcal{F}_{\text{esc}} = \mathcal{C}_{\text{ana}} \wedge \neg \mathcal{C}_{\text{num}}.$$

- **Recuperados:** entre os falsos escuros, aqueles que se tornam estelares após a correção:

$$\mathcal{R} = \mathcal{F}_{\text{esc}} \wedge \mathcal{C}_{\text{corr}}.$$

- **Falsos estelares:** halos que são escuros no analítico, mas estelares no numérico bruto:

$$\mathcal{F}_{\text{est}} = \neg \mathcal{C}_{\text{ana}} \wedge \mathcal{C}_{\text{num}}.$$

- **Acurácia numérica bruta:**

$$A_{\text{num}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathcal{C}_{\text{num}}^{(i)} = \mathcal{C}_{\text{ana}}^{(i)}] \times 100\%.$$

- **Acurácia corrigida:**

$$A_{\text{corr}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathcal{C}_{\text{corr}}^{(i)} = \mathcal{C}_{\text{ana}}^{(i)}] \times 100\%.$$

- **Taxa de recuperação:**

$$R = \frac{|\mathcal{R}|}{|\mathcal{F}_{\text{esc}}|} \times 100\%.$$

3.8.4 Intervalo de confiança de Wilson para a taxa de recuperação

Para quantificar a incerteza estatística da taxa de recuperação, o código utiliza o intervalo de confiança de Wilson para proporções binárias. Seja $k = |\mathcal{R}|$ o número de sucessos (recuperados) e $n = |\mathcal{F}_{\text{esc}}|$ o número total de tentativas (falsos escuros). A proporção observada é $\hat{p} = k/n$. O intervalo de confiança de 95% é calculado por

$$\begin{aligned} \text{centro} &= \hat{p} + \frac{z^2}{2n}, \quad \text{margem} = z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}, \quad \text{denom} = 1 + \frac{z^2}{n}, \\ \text{IC}_{95\%} &= \left(\frac{\text{centro} - \text{margem}}{\text{denom}}, \frac{\text{centro} + \text{margem}}{\text{denom}} \right), \end{aligned}$$

com $z = 1.96$ para 95% de confiança. Esta fórmula é implementada na função interna `_wilson_ic`.

3.8.5 Erros do potencial

Além das métricas de classificação, a função calcula o erro relativo do potencial em relação ao analítico, antes e depois da correção, para cada halo:

$$\delta_{\text{num}}^{(i)} = \left| \frac{\Phi_{\text{num}}^{(i)} - \Phi_{\text{ana}}^{(i)}}{\Phi_{\text{ana}}^{(i)}} \right| \times 100, \quad \delta_{\text{corr}}^{(i)} = \left| \frac{\Phi_{\text{corr}}^{(i)} - \Phi_{\text{ana}}^{(i)}}{\Phi_{\text{ana}}^{(i)}} \right| \times 100.$$

A partir destes vetores, calculam-se a média, o máximo e o percentil 95 do erro para cada método. O percentil 95 é particularmente útil, pois indica que 95% dos halos apresentam erro inferior a este valor, sendo mais robusto que o máximo (suscetível a *outliers*) para caracterizar a precisão típica do método.

A implementação completa da função `monte_carlo` é apresentada a seguir, com a estrutura do loop de classificação e o cálculo das métricas finais.

```
1 def monte_carlo(corretor, N_halos=100, N_grade=128, M_sub=8, seed=RNG_SEED,
```

```

2         N_min_estelar=30):
3
4         # Amostra-base uniforme
5         log_M_base, c_base = _sorteia(N_halos, rng, estrato=False)
6         n_estelar_base = sum(_classifica_analitico(lm, cc)
7                               for lm, cc in zip(log_M_base, c_base))
8
9         log_M_extra_list, c_extra_list = [], []
10        n_estelar_total = n_estelar_base
11        while n_estelar_total < N_min_estelar and tentativas < max_tentativas:
12            lm_e, c_e = _sorteia(1, rng_extra, estrato=True)
13            if _classifica_analitico(lm_e[0], c_e[0]):
14                n_estelar_total += 1
15            log_M_extra_list.append(lm_e[0]); c_extra_list.append(c_e[0])
16
17        for i in range(N_halos_efetivo):
18
19
20        falso_escuro = class_a & ~class_n
21        recuperados = falso_escuro & class_c
22        falso_estelar = ~class_a & class_n
23        pct_rec = 100.0 * recuperados.sum() / max(falso_escuro.sum(), 1)
24        acc_n = 100.0 * (class_n == class_a).sum() / N_halos_efetivo
25        acc_c = 100.0 * (class_c == class_a).sum() / N_halos_efetivo
26
27        ic_lo, ic_hi = _wilson_ic(int(recuperados.sum()), int(falso_escuro.sum()
28                                ()))
29
30        delta_num_pct = np.abs(phi_ana-phi_num_)/np.abs(phi_ana)*100
31        delta_corr_pct = np.abs(phi_ana-phi_corr)/np.abs(phi_ana)*100
32
33        return dict(...)

```

Listing 14: Função `monte_carlo` para simulação Monte Carlo de halos

Os resultados desta simulação Monte Carlo são consolidados em uma tabela de auditoria exportada pela função `exportar_csv`, contendo 21 colunas por halo, incluindo identificadores, parâmetros físicos, potenciais, scores, classificações, flags de diagnóstico e a coluna `estrato` que distingue a amostra-base da amostra estratificada. As métricas agregadas (taxa de recuperação com intervalo de confiança de Wilson, acurácias e erros do potencial) são impressas na saída padrão e também armazenadas no dicionário de retorno para posterior análise e visualização.

3.9 Etapa 8 – Fração de ocupação de galáxias

A etapa anterior estabeleceu a classificação individual de cada halo da amostra Monte Carlo como estelar ou escuro, segundo os três métodos (analítico, numérico bruto e numérico corrigido). Para extrair uma informação populacional que possa ser diretamente comparada com observações e com modelos teóricos de formação de galáxias, o código implementa a função `calcular_focc`, que calcula a *fração de ocupação de galáxias* (galaxy occupation fraction, f_{occ}) em função da velocidade máxima do halo, V_{max} .

A fração de ocupação de galáxias é definida como a probabilidade de um halo de matéria escura com uma dada velocidade máxima hospedar uma galáxia (ou seja, ser classificado como estelar). Formalmente:

$$f_{\text{occ}}(V_{\text{max}}) = P(\text{estelar} \mid V_{\text{max}}).$$

Na prática, para um conjunto finito de halos, a f_{occ} é estimada como a fração de halos classificados como estelares dentro de cada intervalo (bin) de V_{max} :

$$\hat{f}_{\text{occ}}^{(j)} = \frac{1}{N_j} \sum_{i \in \mathcal{B}_j} \mathbf{1}[\mathcal{C}^{(i)} = \text{estelar}],$$

onde $\mathcal{B}_j$ é o conjunto de halos no j -ésimo bin de V_{max} , $N_j = |\mathcal{B}_j|$ é o número de halos neste bin, e $\mathbf{1}[\cdot]$ é a função indicadora que vale 1 se a condição for verdadeira e 0 caso contrário.

3.9.1 Definição dos bins de V_{max}

A função `calcular_focc` recebe como entrada o dicionário de resultados do Monte Carlo (`mc`) e o número de bins N_{bins} (padrão 12). O intervalo de V_{max} é definido a partir dos valores mínimo e máximo da amostra, com uma pequena expansão para garantir que todos os pontos sejam incluídos:

$$V_{\text{min}} = 0.9 \times \min_i V_{\text{max}}^{(i)}, \quad V_{\text{max}} = 1.05 \times \max_i V_{\text{max}}^{(i)}.$$

Os limites dos bins são igualmente espaçados:

$$v_j = V_{\text{min}} + j \Delta v, \quad \Delta v = \frac{V_{\text{max}} - V_{\text{min}}}{N_{\text{bins}}}, \quad j = 0, \dots, N_{\text{bins}}.$$

Os centros dos bins são definidos como

$$v_{\text{cent}}^{(j)} = \frac{v_j + v_{j+1}}{2}, \quad j = 0, \dots, N_{\text{bins}} - 1.$$

3.9.2 Cálculo da fração de ocupação

Para cada método de classificação (analítico, numérico e corrigido), a função calcula a fração de ocupação em cada bin:

$$\hat{f}_{\text{occ}, \text{mtodo}}^{(j)} = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathcal{C}_{\text{mtodo}}^{(i)},$$

onde $\mathcal{C}_{\text{mtodo}}^{(i)}$ é a classificação binária do i -ésimo halo no j -ésimo bin (1 para estelar, 0 para escuro). Bins com menos de dois halos ($N_j < 2$) são descartados e recebem valor NaN, pois a estimativa da fração seria estatisticamente instável.

```

1 def calcular_focc(mc, N_bins=12):
2     Vm = mc["Vmax"]
3     bins = np.linspace(Vm.min()*0.9, Vm.max()*1.05, N_bins+1)
4     bc = 0.5*(bins[:-1]+bins[1:])
5
6     def _focc_bin(cls):
7         fo = np.full(N_bins, np.nan)
8         for j, (v0, v1) in enumerate(zip(bins[:-1], bins[1:])):
9             m = (Vm >= v0) & (Vm < v1)
10             if m.sum() >= 2:
11                 fo[j] = cls[m].mean()
12         return fo
13
14     fo_a = _focc_bin(mc["class_a"])
15     fo_n = _focc_bin(mc["class_n"])
16     fo_c = _focc_bin(mc["class_c"])
17
18     return dict(bins=bins, bin_centers=bc,
19                 focc_analitico=fo_a, focc_numerico=fo_n, focc_corrigido=
20                 fo_c)

```

Listing 15: Função `calcular_focc` para cálculo da fração de ocupação

3.9.3 Modelo logístico para a curva de ocupação

A curva de ocupação esperada para o critério de Nadler tem a forma de uma sigmóide, com transição abrupta na vizinhança do limiar $V_{\text{max}} \approx 12$ km/s. Para caracterizar quantitativamente esta curva e extrair parâmetros físicos, o código define a função logística

$$f_{\log}(x; x_0, k) = \frac{1}{1 + \exp[-k(x - x_0)]},$$

onde:

- x_0 é o ponto de inflexão da curva, que corresponde ao valor de V_{max} para o qual $f_{\text{occ}} = 0.5$;

- k é a taxa de inclinação (parâmetro de forma), que controla a abruptidade da transição: quanto maior k , mais abrupta é a transição entre halos escuros e estelares.

A função é implementada conforme a definição matemática:

```
1 def _logistica(x, x0, k):
2     return 1.0 / (1.0 + np.exp(-k * (x - x0)))
```

Listing 16: Função logística

Embora a função `calcular_focc` não realize o ajuste logístico automaticamente, a presença da função `_logistica` no código indica que a análise pode ser estendida para extrair os parâmetros x_0 e k para cada um dos três métodos, utilizando, por exemplo, a função `curve_fit` da biblioteca SciPy. Os parâmetros obtidos permitem quantificar o deslocamento do limiar devido ao viés numérico:

$$\Delta x_0 = x_0^{\text{num}} - x_0^{\text{ana}},$$

e verificar se a correção PCHIP restaura o valor correto:

$$x_0^{\text{corr}} \approx x_0^{\text{ana}}.$$

3.9.4 Interpretação física e validação

A curva de ocupação $f_{\text{occ}}(V_{\text{max}})$ é uma ferramenta fundamental em modelos de formação de galáxias, conectando a população de halos de matéria escura (prevista pela cosmologia) à população de galáxias observada. No contexto deste trabalho, a comparação entre as três curvas ($f_{\text{occ}}^{\text{ana}}$, $f_{\text{occ}}^{\text{num}}$ e $f_{\text{occ}}^{\text{corr}}$) serve como um teste visual e quantitativo da eficácia do corretor PCHIP:

- A curva analítica $\hat{f}_{\text{occ}}^{\text{ana}}(V_{\text{max}})$ deve apresentar uma transição abrupta centrada em $V_{\text{max}} \approx 12$ km/s, refletindo o limiar físico de reionização;
- A curva numérica bruta $\hat{f}_{\text{occ}}^{\text{num}}(V_{\text{max}})$ deve apresentar um deslocamento sistemático para valores mais altos de V_{max} (curva deslocada para a direita), pois o viés numérico subestima o potencial, exigindo uma velocidade máxima maior para que o halo seja classificado como estelar;
- A curva corrigida $\hat{f}_{\text{occ}}^{\text{corr}}(V_{\text{max}})$ deve sobrepor-se à curva analítica, demonstrando que a correção PCHIP restaura a posição correta do limiar.

A restauração do ponto de inflexão x_0 para o valor físico ($x_0 \approx 12$ km/s) e a redução da diferença Δx_0 são indicadores quantitativos do sucesso da correção. A função `calcular_focc` fornece os dados necessários para a construção da figura que compara as três curvas, que constitui um dos principais resultados visuais do trabalho.

3.10 Etapa 9 – Geração de figuras

Para comunicar visualmente os resultados da análise e das validações, o código implementa a função `figura_diagnostico`, que gera uma figura composta por seis painéis (dispostos em uma grade 3×2) resumindo todo o processo de calibração, validação e correção. A figura é produzida com o tema cosmológico escuro (fundo escuro, texto claro) definido pela função auxiliar `_ax_dark`, que configura cada eixo com as cores globais BG1, TXT e BDR estabelecidas na Seção A.

Os seis painéis são descritos a seguir.

3.10.1 Painel [0,0]: Perfis de potencial NFW analíticos

Este painel apresenta a família de curvas de potencial $\Phi(r)$ para as cinco concentrações calibradas $c \in \{5, 10, 15, 20, 30\}$, calculadas analiticamente pela classe `NFWAnalitico`. A curva para cada concentração é traçada com a cor correspondente da paleta `CORES_C`, e uma linha vertical marca a posição do raio de avaliação $R_{\text{eval}} = 0.10$. Uma anotação destaca a expansão de Taylor utilizada no código para $r \rightarrow 0$:

$$\Phi(0) = -\frac{c}{f(c)}.$$

3.10.2 Painel [0,1]: Viés $\varepsilon(r)$ antes da correção

Este painel exibe o viés médio $\bar{\varepsilon}(r)$ (em porcentagem) para cada concentração, juntamente com a banda de $\pm 1\sigma$ obtida a partir das translações aleatórias do mapeamento de erro da Etapa 3. Uma faixa vermelha translúcida destaca a região de viés típico de 13–20%, e uma linha vertical marca R_{eval} . Este painel demonstra a magnitude do erro sistemático introduzido pela discretização CIC-FFT antes da aplicação do corretor.

3.10.3 Painel [1,0]: Validação cruzada

Este painel apresenta as quatro curvas de potencial da validação cruzada (Etapa 5): o potencial analítico (linha branca), o potencial de referência com $N_{\text{ref}} = 128$ (linha laranja), o potencial com $N_{\text{cal}} = 64$ sem correção (linha vermelha) e o potencial com $N_{\text{cal}} = 64$ corrigido pelo PCHIP (linha verde). A sobreposição da curva verde com as curvas branca e laranja demonstra visualmente a eficácia da correção.

3.10.4 Painel [1,1]: Erro antes e depois da correção

Este painel exibe o erro relativo percentual $\varepsilon(r)$ (em relação ao analítico) para a grade de baixa resolução, antes (linha vermelha) e depois (linha verde) da correção. Uma faixa sombreada em verde indica a meta de precisão de $\pm 1\%$. Uma caixa de texto no canto superior

direito resume a redução do erro médio e os valores quantitativos antes e depois da correção. A linha vertical em R_{eval} permite avaliar o erro no raio de interesse físico.

3.10.5 Painel [2,0]: Scatter Φ_{ana} vs. $\Phi_{\text{num}} / \Phi_{\text{corr}}$

Este painel apresenta um gráfico de dispersão para todos os halos da simulação Monte Carlo. No eixo horizontal está o potencial analítico Φ_{ana} ; no eixo vertical, os potenciais numérico bruto (triângulos) e corrigido (círculos). Cada ponto é colorido pela concentração do halo (escala de cores `plasma`). A linha tracejada branca representa a identidade $\Phi_{\text{num}} = \Phi_{\text{ana}}$. Os halos que foram recuperados (falsos escuros que se tornaram estelares após a correção) são destacados com estrelas brancas. Este painel permite avaliar a correção *halo a halo*.

3.10.6 Painel [2,1]: Tabela de confusão

Este painel exibe uma tabela de confusão que resume as classificações dos halos para os três métodos: analítico, numérico bruto e corrigido. A tabela apresenta:

- O número de halos estelares e escuros para cada método;
- A acurácia (em porcentagem) do método numérico bruto e do corrigido em relação ao analítico;
- O número de falsos escuros (halos estelares no analítico, escuros no numérico bruto);
- O número de halos recuperados pela correção, juntamente com a taxa de recuperação R e seu intervalo de confiança de Wilson (calculado na Etapa 7).

3.10.7 Estilo visual e salvamento

A função aplica o tema escuro a todos os eixos, ajusta cores, grades e legendas, e salva a figura em formato PNG (150 dpi) e PDF, com fundo preto e título geral descrevendo o conteúdo. O código da função é apresentado a seguir em sua estrutura principal.

```

1 def figura_diagnostico(res_mapa, validacao, mc, saida=None):
2     fig = plt.figure(figsize=(16, 18), facecolor=BG0)
3     gs = gridspec.GridSpec(3, 2, fig, hspace=0.44, wspace=0.36,
4                             left=0.08, right=0.96, top=0.93, bottom=0.06)
5
6     # Painel [0,0]: Perfis NFW
7     ax = _ax_dark(fig.add_subplot(gs[0,0]))
8     ...
9
10    ax = _ax_dark(fig.add_subplot(gs[0,1]))
11    ...
12
13    ax = _ax_dark(fig.add_subplot(gs[1,0]))

```

```

14     ...
15
16     # Paine1 [1,1]: Erro antes vs depois
17     ax = _ax_dark(fig.add_subplot(gs[1,1]))
18     ...
19
20     # Paine1 [2,0]: Scatter
21     ax = _ax_dark(fig.add_subplot(gs[2,0]))
22     ...
23
24     ax = _ax_dark(fig.add_subplot(gs[2,1]))
25     ...
26
27     fig.suptitle(...)
28     plt.savefig(saida, dpi=150, bbox_inches="tight", facecolor=BG0)
29     plt.close(fig)

```

Listing 17: Estrutura da função `figura_diagnostico`

3.11 Etapa 10 – Auditoria e exportação de dados

Para garantir a rastreabilidade e a reprodutibilidade da análise, o código implementa rotinas de exportação de dados que salvam em formato CSV todas as informações relevantes da simulação Monte Carlo e dos estudos de robustez.

3.11.1 Tabela de auditoria completa

A função `exportar_csv` recebe o dicionário de resultados do Monte Carlo e um caminho de arquivo, e exporta uma tabela com 21 colunas para cada halo. As colunas estão organizadas nos seguintes grupos:

- **Identificação e parâmetros:** `halo_id`, `log10_Mvir`, `Mvir_Msol`, `c_NFW`;
- **Velocidades:** `Vvir_kms`, `Vmax_kms`;
- **Potenciais:** `phi_ana`, `phi_num`, `phi_corr`;
- **Scores de Nadler:** `score_ana`, `score_num`, `score_corr`;
- **Classificações:** `class_ana`, `class_num`, `class_corr`;
- **Diagnóstico:** `falso_escuro`, `recuperado`, `falso_estelar`;
- **Erros relativos:** `delta_phi_num_pct`, `delta_phi_corr_pct`;
- **Estrato amostral:** `estrato` (0 para amostra-base, 1 para estratificada).

A coluna `estrato` é particularmente importante, pois permite que o leitor ou usuário do código reanalise os dados considerando apenas a amostra-base uniforme, se desejar, isolando o efeito da estratificação complementar.

Os erros relativos são calculados como

$$\delta_{\text{num}} = \frac{\Phi_{\text{num}} - \Phi_{\text{ana}}}{|\Phi_{\text{ana}}|} \times 100, \quad \delta_{\text{corr}} = \frac{\Phi_{\text{corr}} - \Phi_{\text{ana}}}{|\Phi_{\text{ana}}|} \times 100,$$

com tratamento especial para evitar divisão por zero ($|\Phi_{\text{ana}}| < 10^{-10}$).

```

1 def exportar_csv(mc, caminho):
2     with open(caminho, "w", newline="", encoding="utf-8") as f:
3         w = csv.DictWriter(f, fieldnames=cols)
4         w.writeheader()
5         for i in range(N):
6
7             w.writerow({...})
8     print(f" CSV exportado: {caminho}  ({N} halos, 21 colunas)")

```

Listing 18: Função `exportar_csv` para auditoria dos halos

3.11.2 Relatório de robustez suplementar

A função `exportar_csv_robustez` consolida os resultados dos estudos de robustez das Etapas 6.5 e 7 em um único arquivo CSV. O relatório contém seções separadas para:

- **Ambiente de execução:** data, versões das bibliotecas e parâmetros da simulação (quando fornecidos);
- **Convergência de M_{sub} :** tabela com M_{sub} , erro central, erro em R_{eval} e tempo de CPU;
- **Sensibilidade ao tamanho da caixa:** tabela com L e erro em R_{eval} ;
- **Sensibilidade a Φ_{ref} :** tabela com c_{ref} , Φ_{ref} , score e classificação para o halo de teste;
- **Projeção de escalabilidade:** tempos medidos para $N = 32, 64, 128$, tempo projetado para $N = 256$, expoente ajustado p e custo total projetado em minutos.

Este relatório atende diretamente às exigências de documentação de convergência numérica, sensibilidade a parâmetros e justificativa das escolhas metodológicas, conforme listado na avaliação.

```

1 def exportar_csv_robustez(caminho, conv_msub=None, sens_lbox=None,
2                           sens_phiref=None, projecao=None, ambiente=None):
3     with open(caminho, "w", encoding="utf-8") as f:
4         f.write("# Estudos de robustez suplementares\n")
5         if ambiente:

```

```

6         f.write("\n[ambiente_execucao]\n")
7         for k, v in ambiente.items():
8             f.write(f"{k}, {v}\n")
9     if conv_msub:
10        f.write("\n[convergencia_M_sub]\n")
11        f.write("M_sub, erro_central_pct, erro_reval_pct, tempo_s\n")
12        ...

```

Listing 19: Estrutura da função `exportar_csv_robustez`

3.11.3 Orquestração do pipeline completo

A função `executar_pipeline_completo` orquestra todas as etapas descritas nas seções anteriores em uma única execução. O fluxo é o seguinte:

1. Calibração do corretor PCHIP através de `mapear_erro` para as cinco concentrações;
2. Execução das validações (cruzada, Hernquist, convergência M_{sub} , sensibilidade L , sensibilidade Φ_{ref} , projeção de escalabilidade), conforme parâmetros booleanos;
3. Execução da simulação Monte Carlo com estratificação;
4. Cálculo da fração de ocupação e ajuste logístico;
5. Exportação dos CSVs de auditoria e robustez;
6. Geração das figuras (incluindo a figura de diagnóstico e a curva de f_{occ});
7. Escrita de um relatório final em texto com as principais métricas e localização dos arquivos gerados.

A função retorna um dicionário contendo o corretor calibrado, os resultados do Monte Carlo, os resultados da f_{occ} e as métricas dos ajustes logísticos, permitindo análise posterior no ambiente interativo.

```

1 def executar_pipeline_completo(N_grade=128, M_sub=8, N_halos=100,
2                               N_min_estelar=30, pasta_saida="./resultados"
3                               ,
4                               seed=RNG_SEED):
5     for c in CONCS_MAP:
6         res = mapear_erro(c, N=N_grade, Ntr=100, M_sub=M_sub, seed=seed+int
7         (c))
8         resultados_calib.append(res)
9         corretor = CorretorPCHIP(resultados_calib)
10        ...

```

```

11     mc_resultados = monte_carlo(corretor, N_halos, N_grade, M_sub, seed
12     +100, N_min_estelar)
13
14     focc_resultados = calcular_focc(mc_resultados)
15
16     exportar_csv(mc_resultados, ...)
17     exportar_csv_robustez(...)
18
19     6. Figuras
20     figura_diagnostico(...)
21     # figura da f_occ
22
23     with open(relatorio, "w") as f:
24         f.write(...)
25
26     return dict(corretor=corretor, mc=mc_resultados, focc=focc_resultados,
27     ...)

```

Listing 20: Estrutura da função `executar_pipeline_completo`

O pipeline completo é projetado para ser executado de forma autônoma, produzindo todos os arquivos necessários para a documentação do trabalho em uma única execução, com os resultados salvos na pasta especificada pelo usuário.

4 Resultados

4.1 Validação analítica dos perfis NFW e Hernquist

A Tabela 1 apresenta os parâmetros de validação analítica para as cinco concentrações estudadas, incluindo o raio de escala NFW (r_s), o potencial central NFW ($\Phi_{0,\text{NFW}}$), a velocidade circular máxima NFW ($V_{\text{m,NFW}}$) e as grandezas equivalentes para o perfil de Hernquist (a_{HQ} , $\Phi_{0,\text{HQ}}$, $V_{\text{m,HQ}}$). O potencial de referência adotado para a classificação estelar/escuro foi $\Phi_{\text{ref}} = -4,6557$, com limiar de *score* $\text{SCORE_LIM} = 0,85$ e limiar de velocidade máxima $V_{\text{MAX_LIM}} = 12,0$ km/s.

Table 1: Validação analítica dos perfis NFW e Hernquist por concentração c

c	$r_{s,\text{NFW}}$	$\Phi_{0,\text{NFW}}$	$V_{\text{m,NFW}}$	a_{HQ}	$\Phi_{0,\text{HQ}}$	$V_{\text{m,HQ}}$
5	0,2000	-5,2169	1,0621	0,0926	-10,8000	1,6432
10	0,1000	-6,7168	1,2051	0,0463	-21,6000	2,3238
15	0,0667	-8,1740	1,3294	0,0309	-32,4000	2,8460
20	0,0500	-9,5596	1,4377	0,0231	-43,2000	3,2863
30	0,0333	-12,1642	1,6218	0,0154	-64,8000	4,0247

O teste de fumaça do solver ($N = 32$, $c = 10$) produziu $\Phi_{\text{num}} = -3,5551$ contra $\Phi_{\text{ana}} =$

−5,3684, erro de 33,8%, compatível com o esperado ($\sim 30\%$) para uma grade grosseira, validando o funcionamento correto do pipeline antes do mapeamento sistemático de erros.

4.2 Mapeamento do erro numérico em função da concentração

O mapeamento do erro espectral, mediado sobre 100 translações sub-célula em grade $N = 128$, revelou uma relação monotônica decrescente entre o erro relativo do potencial e a concentração c do halo (Tabela 2). Halos mais concentrados (perfis mais “pontudos”) apresentam menor erro relativo de suavização, mas ainda assim substancial (15,5% a 38,9% na célula central).

Table 2: Erro relativo do potencial numérico (sem correção) em função da concentração, grade $N = 128$, 100 translações

c	$\varepsilon(r \rightarrow 0)$	$\varepsilon(r_{\text{eval}})$	Máx.	P95	IC95%
5	$38,9 \pm 0,0\%$	46,5%	46,5%	46,5%	[46,5; 46,5]%
10	$27,1 \pm 0,0\%$	36,5%	36,5%	36,5%	[36,5; 36,5]%
15	$21,8 \pm 0,1\%$	31,9%	31,9%	31,9%	[31,9; 31,9]%
20	$18,8 \pm 0,1\%$	29,2%	29,3%	29,3%	[29,2; 29,2]%
30	$15,5 \pm 0,1\%$	26,2%	26,2%	26,2%	[26,2; 26,2]%

Nota-se que o erro no raio de avaliação de referência ($r_{\text{eval}} = 0,1 r_{\text{vir}}$) é sistematicamente superior ao erro na célula central em todas as concentrações testadas, indicando que o viés de suavização não se restringe à cúspide mas se propaga para a região relevante ao critério de Nadler. A intervalo de confiança de 95% praticamente degenerado (largura $\approx 0,0-0,1$ p.p.) reflete a baixíssima variância residual após a média sobre 100 translações independentes, confirmando que o viés é predominantemente sistemático (dependente de r e c) e não estocástico (dependente da fase da malha).

A partir desse mapeamento, construiu-se o operador `CorretorPCHIP` bidimensional, calibrado sobre a grade de concentrações $c = [5, 10, 15, 20, 30]$.

4.3 Validação cruzada do operador de correção

A validação cruzada, com calibração em $N = 128$ e aplicação em grade independente $N = 64$ ($c = 10, 0$), constitui o teste mais rigoroso de generalização do método (Tabela 3). A redução do erro na métrica principal foi de 98,9%. É particularmente relevante o resultado no raio $r_{\text{eval}} = 0,1 r_{\text{vir}}$ – a métrica mais diretamente ligada ao critério de Nadler (2025) – em que o erro caiu de 37,02% para 0,04%, uma melhoria de mais de três ordens de grandeza.

4.4 Teste de sensibilidade ao perfil: transferabilidade para Hernquist

O teste de generalidade com o perfil de Hernquist ($c_{\text{equiv}} = 10, 0$, $N = 64$) revelou uma limitação importante do método: o corretor calibrado para NFW, quando aplicado sem recalibração

Table 3: Validação cruzada NFW: erro do potencial antes e depois da correção PCHIP ($N_{\text{ref}} = 128$, $N_{\text{cal}} = 64$, $c = 10, 0$)

Métrica	Antes	Depois
Erro (referência única)	48,01%	0,51%
Erro máximo (todas as cascas)	85,37%	4,72%
Erro P95 (todas as cascas)	80,72%	1,52%
Erro em $r_{\text{eval}} = 0, 1$ (crítico p/ Nadler)	37,02%	0,04%

a um perfil de Hernquist, **piorou** o erro relativo, de 27,5% (sem correção) para 37,99% (com correção), correspondendo a uma eficiência de -38% . Esse resultado indica que o operador PCHIP, tal como calibrado, é fortemente específico ao perfil NFW (sobreajustado à forma funcional da cúspide $\rho \propto r^{-1}$ na região interna) e não constitui uma correção genérica de *softening* numérico aplicável a qualquer perfil de densidade. A extensão do método a outras famílias de perfis exigirá recalibração independente ou uma parametrização mais geral da função de transferência.

4.5 Estudos de robustez

4.5.1 Convergência em função da subamostragem CIC

A Tabela 4 mostra a convergência do erro relativo em função do parâmetro de subamostragem M_{sub} (grade $N = 32$, $c = 10, 0$).

Table 4: Convergência do erro em função de M_{sub} (grade $N = 32$, $c = 10, 0$)

M_{sub}	sv/célula	$\varepsilon_{\text{central}}$	$\varepsilon(r_{\text{eval}})$	t (s)
2	8	28,50%	35,74%	0,02
4	64	30,88%	36,36%	0,04
8	512	31,51%	36,52%	0,11
16	4096	31,55%	36,52%	0,69

O erro no raio de avaliação praticamente satura entre $M_{\text{sub}} = 8$ e $M_{\text{sub}} = 16$ (variação de apenas 0,00 ponto percentual), confirmando que $M_{\text{sub}} = 8$ (512 subvoxels por célula), adotado nas demais etapas do projeto, oferece precisão de deposição CIC suficiente sem custo computacional adicional injustificado.

4.5.2 Sensibilidade ao tamanho da caixa de simulação

A Tabela 5 evidencia forte sensibilidade do erro ao tamanho da caixa de simulação L (em unidades de r_{vir}), grade $N = 32$.

O aumento expressivo do erro com L é consistente com a perda de resolução espacial efetiva por célula ($dx = L/N$) quando a caixa é ampliada sem aumento proporcional de N : para L fixo

Table 5: Sensibilidade do erro ao tamanho da caixa L (grade $N = 32$, $c = 10, 0$)

$L(r_{\text{vir}})$	$\varepsilon(r_{\text{eval}})$
2,0	36, 36%
3,0	69, 33%
4,0	82, 69%

em número de células, o raio de avaliação (fixado em unidades de r_{vir}) passa a corresponder a um número menor de células de grade, agravando a suavização numérica. Esse resultado reforça a importância de reportar explicitamente a razão r_{eval}/dx em qualquer comparação de vies numérico entre simulações.

4.5.3 Fragilidade da classificação estelar/escuro frente ao potencial de referência

Um dos resultados mais relevantes deste projeto para a interpretação de trabalhos que aplicam o critério de Nadler et al. (2025) é a sensibilidade da classificação binária (estelar *versus* escuro) à escolha, em princípio arbitrária, da concentração de referência c_{ref} utilizada para fixar Φ_{ref} . Para um halo de teste fixo ($c = 8, 0$), variou-se apenas c_{ref} (Tabela 6).

Table 6: Sensibilidade da classificação estelar/escuro ao potencial de referência (halo de teste $c = 8, 0$ fixo)

c_{ref}	Φ_{ref}	$Score$	Classificação
5,0	-4,2305	1,062	Estelar
10,0	-4,6557	0,965	Estelar
15,0	-4,9932	0,900	Estelar
20,0	-5,2511	0,856	Estelar
30,0	-5,6211	0,799	Escuro

O mesmo halo, com propriedades físicas idênticas, muda de classificação (estelar $\leftrightarrow$ escuro) unicamente em função da escolha arbitrária da concentração de referência adotada para o cálculo de Φ_{ref} . Esse resultado confirma, de forma quantitativa, a fragilidade metodológica apontada na avaliação crítica do critério de Nadler (Seção 2.5 da proposta original deste projeto) e sugere que trabalhos futuros que apliquem esse critério devem reportar explicitamente a sensibilidade da classificação à normalização de referência adotada, sob pena de resultados populacionais (frações de halos estelares) não serem diretamente comparáveis entre estudos com convenções distintas.

4.5.4 Escalabilidade computacional

A Tabela 7 apresenta o tempo de execução empírico por halo em função da resolução de grade N .

Table 7: Tempo de execução por halo em função da resolução de grade

N	t_{halo} (s)
32	0,105
64	2,248
128	19,189

O ajuste em escala log-log resultou em um expoente empírico $p = 3,76$, superior ao expoente teórico $p = 3,00$ esperado para um algoritmo FFT ideal em N^3 pontos de grade (o custo teórico da FFT propriamente dita escala como $N^3 \log N$, mas o excesso observado – de N^3 para $N^{3,76}$ – é atribuído majoritariamente à sobrecarga de deposição CIC com $M_{\text{sub}} = 8$ fixo, cujo custo cresce com o número de células de superfície do halo, e a efeitos de gerenciamento de memória em Python/NumPy para arrays de N^3 elementos complexos). A projeção para $N = 256$ resulta em $t_{\text{halo}} \approx 302,4$ s, implicando um custo total de calibração e Monte Carlo da ordem de 3.024 minutos ($\approx 50,4$ horas de CPU) – uma limitação prática relevante para a extensão do método a estudos com resolução mais fina ou amostras Monte Carlo substancialmente maiores.

4.6 Simulação de Monte Carlo: reclassificação de halos

A amostra Monte Carlo final compreendeu 169 halos (100 da amostragem-base uniforme e 69 adicionados por amostragem estratificada na zona crítica de transição, necessária para obter um número mínimo de 30 halos estelares segundo o critério analítico). A Tabela 8 resume o desempenho comparativo dos três métodos de cálculo do potencial (analítico – padrão-ouro; numérico $N = 128$ sem correção; e numérico $N = 128$ com correção PCHIP) na tarefa de classificação estelar/escuro.

Table 8: Desempenho comparativo de classificação estelar/escuro (amostra de 169 halos)

Método	Estelar	Escuro	Acurácia
Analítico (padrão-ouro)	30	139	100,0%
Numérico $N = 128$ (sem correção)	3	166	84,0%
Numérico $N = 128 + \text{PCHIP Gold}$	30	139	100,0%

Sem correção, o solver numérico classificou apenas 3 dos 30 halos verdadeiramente estelares como tal – ou seja, 27 halos estelares genuínos foram classificados erroneamente como “falsos escuros” unicamente por efeito do viés numérico de suavização do potencial. Após a aplicação do operador PCHIP, os 27 halos foram corretamente recuperados (recuperação de 100%, intervalo de confiança de Wilson a 95% de [88%, 100%]), elevando a acurácia global de classificação de 84,0% para 100,0% – igualando o resultado do método analítico de referência.

Em termos de erro absoluto do potencial na amostra Monte Carlo completa, obteve-se: erro médio de 30,4% (máximo 37,0%; P95 34,1%) sem correção, contra erro médio de 0,92% (máximo 1,34%; P95 1,17%) com a correção PCHIP aplicada.

4.7 Impacto na fração de ocupação de galáxias

A velocidade máxima característica de 50% de ocupação estelar (V_{50}) apresentou os seguintes valores: $V_{50,\text{analítico}} = 12,15$ km/s (compatível com o valor esperado de $\approx 12,0$ km/s); $V_{50,\text{numrico}} = 16,02$ km/s, correspondendo a um deslocamento sistemático de $+3,88$ km/s induzido puramente pelo viés numérico; e $V_{50,\text{corrigido}} = 12,15$ km/s, com desvio residual de $+0,00$ km/s em relação ao valor analítico – ou seja, a correção PCHIP eliminou completamente, dentro da precisão numérica reportada, o viés sistemático na determinação da fração de ocupação de galáxias.

4.8 Figuras e produtos de auditoria gerados

O pipeline gerou quatro figuras de nível de publicação: *Figura 1* (diagnóstico geral do método), *Figura 2* (comparação “antes/depois” da correção – *the punch*), *Figura 3* (fração de ocupação de galáxias) e *Figura 4* (teste de sensibilidade Hernquist), além de dois arquivos CSV de auditoria completa: `resultados_mc_gold.csv` (169 halos, 21 colunas, incluindo a coluna de estrato de amostragem) e `robustez_suplementar.csv` (estudos de convergência M_{sub} , sensibilidade L e sensibilidade Φ_{ref}).

5 Discussão

Os resultados obtidos atendem integralmente aos critérios de sucesso estabelecidos nos objetivos específicos do projeto. O erro médio do potencial numérico, tipicamente entre 15,5% e 46,5% na faixa de concentrações estudada, foi reduzido para valores inferiores a 1% em validação cruzada com grade independente – superando com folga o critério de sucesso de erro médio $< 5\%$ para $r \geq 2\Delta x$ estabelecido na proposta original. A aplicação à amostra Monte Carlo demonstrou que esse viés não é uma curiosidade puramente numérica: ele é suficiente para alterar drasticamente (de 100% para 10%) a fração de halos corretamente identificados como capazes de formar estrelas em uma amostra realista, com implicações diretas para previsões de contagem de galáxias satélite.

Por outro lado, dois resultados de robustez qualificam de forma importante o alcance do método. Primeiro, a falha de generalização para o perfil de Hernquist (eficiência de -38%) demonstra que a correção PCHIP, tal como calibrada, constitui uma solução específica para a forma funcional do perfil NFW e não um corretor genérico de *softening* de grade – resultado consistente com a natureza empírica (não puramente teórica) da calibração adotada, que captura características espectrais próprias da cúspide $\rho \propto r^{-1}$ do NFW. Segundo, e talvez mais relevante para a comunidade que aplica o critério de Nadler et al. (2025), a demonstração de que a classificação binária estelar/escuro de um mesmo halo pode se inverter unicamente pela escolha da concentração de referência usada para normalizar Φ_{ref} expõe uma fragilidade metodológica do critério em si, independente de qualquer viés numérico de grade. Esse achado é uma con-

tribuição secundária relevante deste projeto e deveria ser considerado em estudos populacionais que utilizem critérios de limiar de formação estelar baseados em profundidade de potencial normalizada.

A escalabilidade computacional (expoente empírico $p = 3,76$) impõe um limite prático claro à extensão do método para amostras Monte Carlo muito maiores ou para resoluções de grade mais finas sem otimizações adicionais (por exemplo, paralelização em GPU, uso de FFTs multigrid ou redução do número de translações de calibração), pontos sugeridos como trabalhos futuros.

6 Limitações

- (i) O método foi validado exclusivamente para halos NFW esféricos e isolados; a extensão a halos triaxiais, com subestrutura ou embutidos em ambiente cosmológico realista está fora do escopo deste relatório.
- (ii) O corretor PCHIP não generaliza para outras famílias de perfis de densidade (demonstrado para Hernquist), exigindo recalibração específica por família de perfil.
- (iii) O efeito de *aliasing* (dobro de modos de alta frequência) foi estimado como inferior a 10% da potência total para k abaixo de metade da frequência de Nyquist, mas não foi corrigido no escopo deste projeto, constituindo limitação assumida (JING, 2005).
- (iv) A classificação estelar/escuro via critério de Nadler et al. (2025) é utilizada exclusivamente como ilustração de sensibilidade ao viés numérico, não como predição cosmológica realista; o critério em si demonstrou sensibilidade não desprezível à normalização de referência adotada (Seção 4.6.3).
- (v) O custo computacional escala com expoente empírico superior ao teórico ($p = 3,76$ versus $p = 3,00$), limitando a aplicação direta do método, sem otimização adicional, a resoluções de grade superiores a $N = 128$ ou a amostras Monte Carlo superiores a algumas centenas de halos.

7 Conclusão

Este projeto quantificou, de forma sistemática e estatisticamente robusta, o viés numérico de suavização do potencial gravitacional central em simulações de mini-halos NFW discretizados por deposição *Cloud-in-Cell* e resolvidos por FFT, e desenvolveu um operador de correção espectral não paramétrico (PCHIP, bidimensional em raio e concentração) capaz de reduzir esse viés em quase duas ordens de grandeza (de 48,0% para 0,51% em validação cruzada independente). A aplicação do operador a uma amostra Monte Carlo de 169 halos sintéticos demonstrou que a correção é suficiente para recuperar 100% dos halos estelares genuínos que

seriam erroneamente classificados como escuros por efeito puramente numérico, e para eliminar um deslocamento sistemático de $+3,88$ km/s na determinação da velocidade característica de ocupação de galáxias (V_{50}).

Os testes de robustez conduzidos, entretanto, revelam que o método não deve ser interpretado como uma correção universal de viés de grade: sua eficácia é específica ao perfil NFW (falhando para o perfil de Hernquist) e sua interação com o critério de classificação de Nadler et al. (2025) expõe uma sensibilidade adicional e independente à escolha da normalização de referência do potencial. Esses achados, somados à análise de escalabilidade computacional, delimitam com precisão o escopo de validade do método e apontam direções concretas para trabalhos futuros: generalização da função de transferência para outras famílias de perfis, correção explícita do efeito de *aliasing*, e otimização computacional (paralelização, redução do custo de deposição CIC) para viabilizar a aplicação em amostras Monte Carlo de maior escala e em populações cosmológicas realistas com subestrutura.

Referências

- CHABANIER, S. et al. Mitigating resolution effects in cosmological simulations of the Lyman-alpha forest. **Journal of Cosmology and Astroparticle Physics**, v. 2021, n. 12, p. 018, 2021.
- DELOS, M. S.; WHITE, S. D. M. The first dark matter haloes: numerical convergence at the resolution limit. **Monthly Notices of the Royal Astronomical Society**, v. 517, n. 4, p. 4736-4751, 2022.
- DUTTON, A. A.; MACCIÒ, A. V. Cold dark matter haloes in the Planck era: mass–concentration relation. **Monthly Notices of the Royal Astronomical Society**, v. 441, n. 4, p. 3359-3374, 2014.
- GARRISON, L. H. et al. Towards accurate and precise power spectra: correcting for the window function. **Monthly Notices of the Royal Astronomical Society**, v. 461, n. 4, p. 4125-4139, 2016.
- HANSEN, P. C. **Rank-Deficient and Discrete Ill-Posed Problems**. Philadelphia: SIAM, 1998.
- HOCKNEY, R. W.; EASTWOOD, J. W. **Computer Simulation Using Particles**. New York: CRC Press, 1988.
- JING, Y. P. Correcting for the Alias Effect in the Press–Schechter Formalism. **The Astrophysical Journal**, v. 620, n. 2, p. 559-563, 2005.
- ŁOKAS, E. L.; MAMON, G. A. Properties of spherical galaxies and clusters with an NFW density profile. **Monthly Notices of the Royal Astronomical Society**, v. 321, n. 1, p. 155-166, 2001.
- NADLER, E. O. et al. Faint Galaxies, Dark Matter, and the Milky Way System. **The Astrophysical Journal**, v. 924, n. 1, p. 15, 2022. (*Updated framework 2025*).
- POWER, C. et al. The inner structure of Λ CDM haloes – I. A numerical convergence study. **Monthly Notices of the Royal Astronomical Society**, v. 338, n. 1, p. 14-34, 2003.
- SCHAUER, A. T. P.; GLOVER, S. C. O.; KLESSEN, R. S. Star formation in the first galaxies: the impact of primordial supersonic stream velocities. **Monthly Notices of the Royal Astronomical Society**, v. 502, n. 4, p. 4993-5005, 2021.
- VITORIO. **Biasnfw: biasnfw.py**. 2023. GitHub repository. Disponível em: <https://github.com/vetorio/Biasnfw/blob/main/biasnfw.py>. Acesso em: 20 ago. 2026.

A Apêndice A – Ambiente computacional e reprodutibilidade

- Sistema operacional: Linux Mint (*release* 11).
- Processador: AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD, 12 CPUs lógicos.
- Linguagem e bibliotecas: Python 3.13.0; NumPy 2.2.2.
- Configuração principal: $N_{\text{map}} = 128$; $N_{\text{transl}} = 100$; $M_{\text{sub}} = 8$; $N_{\text{halos,base}} = 100$ (+69 estratificados = 169 totais).
- Semente aleatória fixa: `seed=42` (reprodutibilidade garantida).
- Tempo total de execução: 232,6 minutos (≈ 13.963 segundos de CPU nesta configuração de hardware; variações de minutos entre execuções em máquinas ou cargas distintas são esperadas e não indicam alteração dos resultados numéricos, que dependem apenas das sementes fixas).
- *Zero-padding*: fator 2 em cada dimensão (volume quadruplicado), com análise de sensibilidade adicional para fatores 1,5 e 2.
- Arquivos de saída gerados: `fig1_diagnostico_gold.png`, `fig2_punch_gold.png`, `fig3_focc_gold.png`, `fig4_hernquist_gold.png`, `resultados_mc_gold.csv` (169 halos, 21 colunas), `robustez_suplementar.csv`.